

Unidad 1: Ingeniería de software en contexto

Introducción a la ingeniería de software. ¿Qué es?

Software

El software es un conjunto de programas en conjunto con toda la documentación requerida para definir, desarrollar y mantener los programas ejecutables que se entregan al cliente (archivos de configuración utilizados para la ejecución, documentos para el usuario, documentos que describen la estructura del sistema), como así también las herramientas utilizadas para la construcción de este.

Otra forma de definir el software es como, la información o conocimiento que se presenta en distintos niveles de abstracción, desde los requerimientos del sistema (abstracto), hasta el código que ejecuta el mismo (detallado).

Ingeniería de Software

Es una disciplina que se encarga de todos los aspectos de la producción del software, desde la primera etapa de especificación del sistema hasta el mantenimiento del sistema después de que se pone en operación.

Esta ingeniería es importante ya que:

- Con mayor frecuencia se requiere producir de manera rápida y económica sistemas confiables.
- Resulta más barato a largo plazo utilizar métodos y técnicas de ingeniería de software para los sistemas de software. La mayoría de los costos consisten en cambiar el software después de que se pone en operación.

Además, la ingeniería de software se apoya en varias realidades, entre ellas:

- En primer lugar, se debe hacer un esfuerzo para entender el problema antes de desarrollar una aplicación de software. Es decir, se debe entender el problema y las necesidades del cliente antes de proponer una solución.
- El diseño es una actividad crucial en el desarrollo de un software porque ayuda en la calidad de software y en la facilidad de mantenimiento.

La ingeniería de software nace tras las crisis del software, en la cual existía (y existe) un conjunto de dificultades o errores ocurridos en la planificación, estimación de los costos, productividad y calidad de un software, debido, principalmente, a la baja eficacia que presentan una gran cantidad de empresas al momento de desarrollarlo.

Estado actual y antecedentes. La crisis del software.

El término crisis del software hace referencia a un conjunto de hechos relativos al software, planteados en la conferencia de OTAN en 1968 por Friedrich Bauer, quien recalco la dificultad para generar software libre de defectos, fácilmente comprensibles y que sean verificables. Sin embargo, este término fue utilizado anteriormente por Dijkstra en El Humilde Programador.

Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos de Software.

Se denomina ciclo de vida a una serie de pasos a través de los cuales un producto o un proyecto progresa en el tiempo. Es decir, que es una representación simplificada de un proceso, el cuál define elementos del proceso (actividades estructurales, productos del trabajo, tareas, etc.) y el flujo del proceso (o flujo de trabajo), que especifica la relación y el orden de dichos elementos. Es decir, son modelos genéricos de los procesos de software; y establecen los criterios que utiliza para determinar si se debe pasar de una tarea a la siguiente. Y son independientes de los procedimientos de cada actividad del ciclo de vida.

Los modelos de proceso pueden incluir las mismas actividades, pero cada uno, desde una perspectiva particular, pondrá más énfasis en algunas respecto a otras y definirán flujos de procesos diferentes.

Además, establece los criterios que se utilizan para determinar cuándo se procede de una tarea a otra.

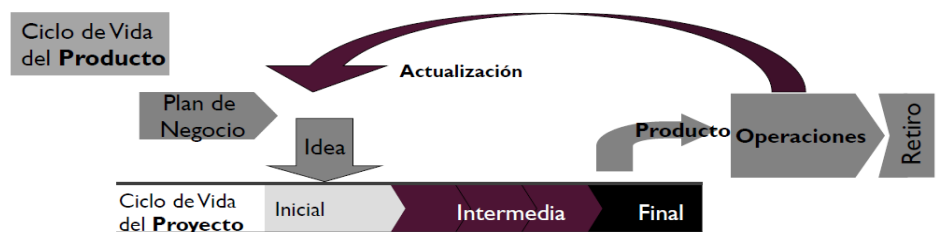
Los modelos de ciclos de vida especifican:

- Las fases del proceso (requerimientos, especificaciones, diseño, etc.)
- Orden en el cual se llevan a cabo.

Relación entre el ciclo de vida del proyecto y del producto

No existe un impacto entre los ciclos de vida de cada uno, sino que el ciclo de vida del producto siempre es mayor que el ciclo de vida del proyecto, ya que el ciclo de vida del proyecto dura lo que dura el desarrollo del software. En cambio, el ciclo de vida del producto dura hasta que el software se deje de utilizar, dependiendo esto de la madurez que tenga el producto en base a las necesidades del mercado.

Es posible que un producto tenga varios proyectos en su ciclo de vida, debido a, constantes cambios y/o actualizaciones que se vayan realizando. Por lo que, dentro del ciclo de vida del producto, se pueden desarrollar varios ciclos de vida de proyectos.



Administración de proyectos dependiendo en ciclo de vida

La elección de un ciclo de vida afecta de forma directa a la administración que se debe realizar sobre el proyecto, ya que cada uno de los modelos de proceso poseen características y enfoques distintos. Por ejemplo, si se realiza un análisis de riesgo y se determina que los requerimientos pueden variar mucho a lo largo del desarrollo, se debería elegir un ciclo de vida en el cual el cambio sea permisible, como los iterativos. En este caso, la administración y la planificación del proyecto se ve afectada en la elección del ciclo de vida.

Además, durante el desarrollo del proyecto, la gestión de este se realiza en consecuencia de esta elección, ya que no es lo mismo gestionar un proyecto que se desarrolla de manera iterativa, que uno que se desarrolla de manera secuencial o en cascada.

Procesos de Desarrollo Empíricos vs. Definidos.

Proceso de desarrollo de software

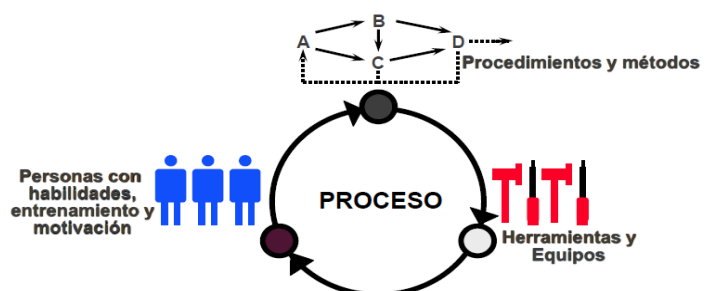
Un proceso es una secuencia de pasos ejecutados para un propósito dado (IEEE), mientras que **un proceso de software se define como un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados.**



Las actividades del proceso de desarrollo son la especificación de requerimientos, el análisis, el diseño, la implementación y la prueba. El orden y la relación entre las diferentes actividades está determinada por la elección del ciclo de vida.

Se tienen tres factores determinantes del proceso:

- **Procedimientos y Métodos:** La adaptación de los procesos establecidos para su implementación, dependiendo del contexto, es esencial para la calidad resultante. Definir las actividades que se harán y cuáles no. Además, es relevante que esta información esté documentada para lograr transparencia. Es decir, tener definidos y escritos los procedimientos.



- **Personas motivadas, capacitadas y con habilidades:** Es factor primordial para obtener la calidad del producto del proceso, ya que éste se determina del esfuerzo de las personas. Sin ellas, no se puede lograr construir ningún

producto. Por ello, deben estar capacitadas y con habilidades para realizar sus tareas asignadas de la manera correcta y motivados, para lograr aún mayor eficiencia. Recordar que el software es una actividad humano-intensiva.

- **Herramientas y Equipos:** Materiales necesarios para llevar a cabo el proceso que nos permiten que las entradas se transformen en salidas. Se recomienda automatizar la mayor cantidad de actividades posibles, lo que mejora la eficiencia de las personas y puedan concentrarse en tareas que requieran de su capacidad.

Los procesos de software son complejos, por lo que no hay un proceso ideal;

Consideraciones

1. Estas actividades **varían dependiendo de la organización y el tipo de sistema que debe desarrollarse**, pero deben incluir: productos, roles, responsabilidades y condiciones
2. **El proceso debe ser explícitamente modelado si va a ser administrado.**
3. **Un proceso de desarrollo se puede aplicar a varios ciclos de vida.** La elección del ciclo de vida depende de la estrategia y este debe permitir seleccionar los artefactos a producir, definir actividades y roles, y modelar conceptos.
4. **Las personas utilizan herramientas y equipos para llevar a cabo los procedimientos** que componen el proceso de desarrollo.

Actividades fundamentales

1. **Especificación de software:** clientes junto con ingenieros definen el software a producir y sus restricciones.
2. **Desarrollo:** diseño y programación del software.
3. **Validación:** verificación para asegurar que es lo que el cliente quiere.
4. **Evolución:** donde se modifica el software para reflejar los requerimientos cambiantes del cliente y mercado.

Procesos Definidos

Los procesos definidos **son considerados deterministas: ante la misma entrada se pretende que se obtendrá la misma salida. Asumen que, si se aplican una y otra vez, se obtendrán siempre los mismos resultados, a pesar de cambiar de equipo.**

1. Están inspirados en las líneas de producción.
2. La administración y el control provienen de la predictibilidad del proceso. Es decir, **el objetivo es lograr una repetibilidad, para poder obtener una previsibilidad de esfuerzo, riesgos, costos, etc.** Asociados al desarrollo de software.
3. **Puedo usar cualquiera de los ciclos de vida.**

En la realidad, no es posible obtener el mismo resultado ante mismas entradas ni con el mismo equipo, ya que depende sumamente del contexto, momento y las personas, por más parecido sean estos factores.

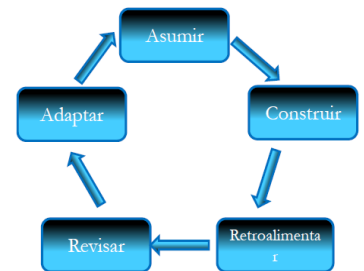
La administración y control del proceso, en muchos de los casos, toman más importancia que el avance del software en sí (más importante la documentación del avance, que el avance en sí).



Procesos Empíricos

Los procesos empíricos se conforman en base a la experiencia interna y externa de las personas que intervienen en un contexto particular. NO es considerado determinista ya que no se asume que ante la aplicación del mismo proceso se obtendrán los mismos resultados. Dado que el factor diferencial es la experiencia sensible, en estos procesos se pueden obtener diferentes resultados dependiendo del contexto en el cual se apliquen.

1. Se basan en ciclos cortos de inspección y adaptación (retroalimentación), porque ante un análisis, se ajustan de mejor forma en aquellos dominios complejos donde prima la creatividad y/o complejidad.
2. La administración y el control es por medio de inspecciones frecuentes y adaptaciones para lograr buenas prácticas.
3. No se pueden combinar con cualquiera de los tipos de ciclos de vida vistos, se suele recomendar que sea el modelo iterativo-incremental, y el que se suele prohibir es el modelo secuencial debido a que no es viable después de 2 años (lo dice una teoría).



La experiencia que se obtiene en cada uno de los procesos no es extrapolable, es decir, que solo funciona en ese mismo proyecto, y se necesita adaptar los procesos a cada contexto en especial. Los conocimientos obtenidos en el proceso se deben hacer explícitos al grupo de desarrollo, y convertir las habilidades individuales en buenas prácticas del trabajo en grupo (compartir conocimiento). Además de la inspección y adaptación, los procesos empíricos poseen un tercer pilar muy importante, que es la transparencia de la información ante todos los integrantes del equipo de trabajo.



Ciclos de vida (Modelos de Proceso) y Procesos de Desarrollo de Software.

Ventajas y desventajas de c/u de los ciclos de vida. Criterios para elección de ciclos de vida en función de las necesidades del proyecto y las características del producto.

Criterios para la selección de un modelo de proceso

Variables a evaluar: Nivel de riesgo, claridad y estabilidad de requerimientos, experiencia técnica del equipo, plazos del proyecto y posibilidad de validación/feedback con el cliente.

1. Si los requerimientos son claros y exhaustivos, en un contexto de baja incertidumbre en el que el cliente no requiere obtener el producto rápidamente, es posible aplicar el modelo secuencial.
2. Si la situación anterior no se presenta y el cliente desea obtener resultados lo antes posible, con las funciones principales del sistema, entonces sería conveniente la elección del modelo iterativo-incremental.
3. Si el objetivo es disminuir los riesgos presentes en el desarrollo de sistemas grandes y complejos, entonces el modelo recursivo puede ser una buena opción.

Ciclo de Vida	Procesos de desarrollo que lo admite	Características que lo hacen elegible
Secuencial	Definidos	• Alta certeza (baja incertidumbre). • La organización tiene una forma tradicional de trabajar. • No se pueden realizar entregas parciales del software (se debe entregar todo junto). • Eliminar riesgos de planificación.
Iterativo/incremental	Definidos o Empíricos	• Existe incertidumbre. • Volatilidad de requerimientos. • Posibilidad de entregas parciales. • Uso en etapas tempranas del producto.
Recurso	Definidos	• Mucho control de riesgos en cada iteración. • No se pueden realizar entregas parciales.

Componentes de un Proyecto de Sistemas de Información.

Proyecto

Es un plan estructurado con inicio y fin orientados a construir una solución digital específica. Consiste en coordinar un equipo multidisciplinario que transforma requisitos y problemas reales en programas funcionales mediante etapas organizadas, gestionando tiempo, presupuesto y herramientas concretas para entregar valor útil a los usuarios finales.

Define el proceso y tareas que se van a realizar, el personal que se encargará de las actividades y los mecanismos que se implementarán para valorar riesgos, controlar el cambio y evaluar la calidad.

Características

Únicos: El resultado de un proyecto será único e irrepetible para cualquier otro proyecto, por más parecido o igual sean sus elementos componentes.

Todos los proyectos por más similares que sean tienen características propias que los hacen únicos, como por ejemplo el calendario (es decir, cronogramas diferentes), o presupuestos.

Orientado a objetivos:

No ambiguos: deben ser lo suficientemente claros para guiar el desarrollo del proyecto.

Medibles: es necesario medir el objetivo para determinar el avance sobre el proyecto.

Alcanzable: deben ser factibles de realizarse por el equipo (que el equipo pueda hacerlos).

Duración limitada de tiempo: Esto implica que un proyecto tiene un principio y un fin, liberando los recursos y personas. Cuando se alcanzan los objetivos del proyecto, este llega a su fin. Un ejemplo claro de lo que no es un proyecto es una línea de producción, ya que esta se ejecuta indefinidamente en el tiempo.

Conjunto de tareas interrelacionadas basadas en esfuerzo y recursos: Se definen las tareas, sus dependencias, los recursos y personas asignadas, permitiendo manejar la complejidad inherente de los sistemas.

Administración de proyecto de software

La administración de proyectos es la organización y coordinación de personas y recursos que permiten desarrollar un producto o servicio de acuerdo con el presupuesto, tiempo y funcionalidades acordadas con el cliente, asegurando que se entregue un producto o servicio de alta calidad. La administración de proyectos es una parte fundamental para que el proyecto sea exitoso. Esta disciplina tiene dos grandes actividades:

- **Planificación.**

- **Monitoreo y control.**

La administración incluye:

1. **Identificar los requerimientos.**
2. **Establecer objetivos claros y alcanzables.**
3. **Adaptar las especificaciones, planes y el enfoque a los diferentes intereses de los involucrados (stakeholders).**

Una buena gestión no puede garantizar el éxito del proyecto. Sin embargo, una mala gestión usualmente implica una falla en el proyecto: el software puede entregarse tarde, costar más de lo estimado originalmente o no cumplir las expectativas de los clientes. (esto se verá mejor explicado en la triple restricción).

Un proyecto planificado puede fracasar también, lo importante de la planificación es el acto de planificar, no el resultado. El acto de ponerme en el ejercicio de pensar tanto: riesgos, objetivos, alcances, estimaciones, etc.

Metas más importantes de la administración de proyectos:

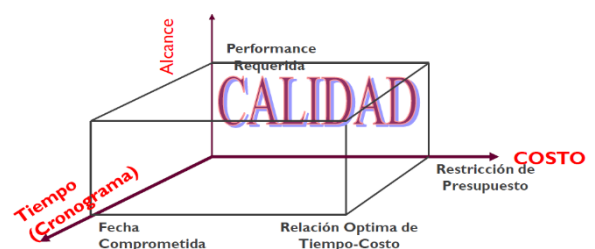
Entregar un software dentro del tiempo acordado, en el presupuesto acordado, cumpliendo con las expectativas del cliente con un equipo optimo que funcione bien

Atributos particulares o complejidades en la administración de proyectos de software:

1. **El producto es intangible:** No poder ver ni tocar el producto que se obtiene al ejecutar el proyecto dificulta la medición del progreso del proyecto. Una construcción civil es visible y se puede evidenciar el progreso en función de las partes construidas.
2. **Los proyectos de software suelen ser excepcionales:** Los conocimientos aprendidos en un proyecto de software son difícilmente extrapolables a otro proyecto, como validación de riesgos, aplicación de nuevas tecnologías, etc. El factor tecnológico es una de las causas.
3. **Los procesos de desarrollo son variables y específicos de la organización:** A pesar del intento de estandarizar los procesos de desarrollo, la realidad es que no es posible predecir de manera confiable qué proceso es el adecuado para adoptar en el proyecto.

La triple restricción – enfoque tradicional

Las variables de restricción de un proyecto son 3: tiempo, costo y alcance. El balance de estos tres factores afecta directamente la calidad del proyecto, ya que un proyecto de alta calidad es aquel que entrega el producto, servicio o resultado requerido, satisfaciendo los objetivos en el tiempo estipulado y con el presupuesto planificado. Es responsabilidad del Líder de proyecto balancear estas variables.



El concepto de la triple restricción hace referencia a que no es posible fijar de forma arbitraria a las 3 variables, sino que **será posible fijar 2 y la tercera se ajustará a lo definido.**

No se negocia la calidad, si no puedo modificar tiempo y/o costo, debido a que es como entregar un software sin probarlo. **La calidad del producto que le entrego al cliente se ve afectada cuando no puedo equilibrar estos 3 factores.**

Alcance – ¿Qué trata de alcanzar?

Son los requerimientos que se incluyen en el proyecto y definen la funcionalidad que tendrá el producto a entregar al cliente. En las metodologías tradicionales el alcance es lo primero que se determina y se deja fijo en función de las

necesidades del cliente, mientras que en ágil se va definiendo, refinando y adaptando en base a las retroalimentaciones del cliente. Esta variable es la que primero se negocia con el cliente.

Tiempo - ¿Cuánto tiempo me llevaría?

Define la fecha de calendario en la cual se compromete a finalizar el proyecto y entregarle el producto resultante al cliente. En las metodologías ágiles, el tiempo es fijo.

Costo - ¿Cuánto debería costar?

Define el costo de todos los recursos implicados en el desarrollo del proyecto. Esto incluye equipamiento, infraestructura, equipos, salarios, entre otros.

En la realidad, el costo y el tiempo del proyecto varían de forma directa con la definición del alcance del producto, es decir, el alcance es directamente proporcional al tiempo y al costo. Cuando el alcance aumenta (mayor cantidad de funcionalidades), el tiempo y dinero (costos) necesarios también deben aumentar, para lograr abordar un proyecto más grande. El problema en la aplicación de esta visión en enfoques tradicionales es que, al utilizar procesos definidos con ciclos de vida en cascada, los requerimientos se definen al comienzo del proyecto, por lo que cualquier modificación o cambio en el alcance del producto final, modifica el tiempo y presupuesto de este. Estos cambios, generan una baja en la calidad del producto final ya que, que se sacrifican funcionalidades o bien, se saltean etapas de testing, etc. debido a que el cliente siempre espera que se entregue lo pactado, respetando el presupuesto y tiempo de entrega.

Líder de Proyecto

En un enfoque tradicional, el líder de proyecto realiza la toma de decisiones en su totalidad sin la inclusión de la opinión del equipo. El líder asigna las tareas que deben realizar los integrantes del equipo, y además, maneja todas las relaciones con todos los stakeholders del proyecto (clientes, gerencias, contratistas, stakeholders en general).

Usualmente los profesionales técnicos no disponen de habilidades necesarias para tratar con la gente, por lo que es necesario una figura de líder de proyecto con las siguientes aptitudes:

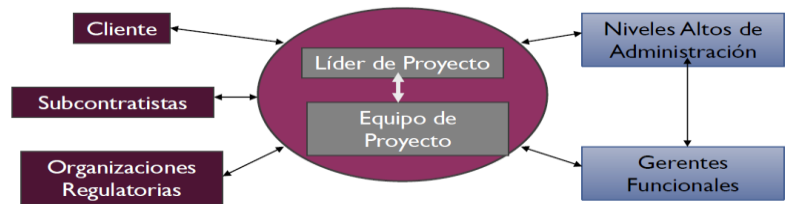
1. Motivación: habilidad para alentar al personal técnico a producir a su máxima capacidad.
2. Organización: habilidad para adaptar el proceso existente (o inventar nuevos), para lograr que el concepto inicial se transforme en el producto final.
3. Innovación: habilidad para alentar a las personas a crear y sentirse creativas.
4. Empatía.
5. Comunicación.
6. Liderazgo.

También cuenta con habilidades duras, relacionadas a los conocimientos del producto, técnicas y herramientas. Dado que el líder de proyecto tiene un enfoque basado en la resolución de problemas, también son características propias las siguientes:

1. Resolución de problemas: identifica conflictos técnicos y organizativos, estructura la solución o motiva a otros profesionales para que la desarrollen, aplica lecciones aprendidas en otros proyectos y adapta el proceso de resolución ante inconvenientes.
2. Identidad administrativa: tiene la confianza de asumir el control cuando es necesario.
3. Logro: recompensa las iniciativas y los logros, incentivando al equipo a correr los riesgos de manera controlada.
4. Influencia: define la estructura del equipo en función de la retroalimentación que él mismo suministra mediante palabras y gestos.

Entre las responsabilidades del líder de proyecto se encuentran, en las nuevas metodologías:

1. Definir el alcance del proyecto.
2. Identificar a los involucrados, recursos y presupuesto.
3. Detallar las tareas, estimar los tiempos y requerimientos.
4. Identificar y evaluar riesgos.
5. Preparar planes de contingencia.
6. Controlar hitos y participar en las revisiones de las fases del proyecto.
7. Administrar el proceso de control de cambios.
8. Producir reportes de estado.



Equipo de proyecto

Es un grupo de personas comprometidas con alcanzar un conjunto de objetivos de los cuales se sienten mutuamente responsables.

Entre sus características se encuentran:

1. Cuentan con conocimientos, habilidades técnicas, motivación, experiencias y diversas personalidades. Además, deben tener un sentimiento de mejora continua y aprendizaje mutuo.
2. Usualmente son un grupo pequeño, esto permiten lograr una comunicación efectiva entre los miembros del equipo.
3. Trabajan en forma conjunto con espíritu de grupo, convirtiéndose en un equipo cohesivo donde prima la sinergia de este. Para eso, se debe lograr un equipo maduro y estable, es decir, que no se agrega o cambia un integrante constantemente.
4. Sentido de responsabilidad como una unidad y se focalizan en el cumplimiento de las metas grupales.

En equipos cohesivos:

1. EL grupo establece sus propios estándares de calidad.
2. Los individuos aprenden de los demás y se apoyan mutuamente.
3. El conocimiento se comparte.
4. Prevalece la mejora continua.

Stakeholders

Son los interesados del proyecto, incluye el equipo de proyecto, el equipo de dirección, el líder de proyecto y el patrocinador.

UNIDAD 2:

Manifiesto Ágil

El desarrollo de software es una actividad empírica y humano-intensiva. Regla de oro para parcial: los elementos de la derecha tienen valor, pero se priorizan los de la izquierda.

Los 4 Valores

1. **Individuos e interacciones sobre procesos y herramientas:** El éxito radica en personas motivadas y comunicación fluida. Los procesos deben adaptarse al equipo, no el equipo a los procesos.
2. **Software funcionando sobre documentación extensiva:** No elimina la documentación, elimina la burocracia inútil. Se documenta solo lo que aporta valor. La retroalimentación real surge de interactuar con el software.
3. **Colaboración con el cliente sobre negociación contractual:** Los contratos rígidos generan fricción ante cambios inevitables. Se busca un cliente integrado al equipo, que pruebe y valide cada incremento.
4. **Respuesta ante el cambio sobre seguir un plan:** Reconoce la incertidumbre. En lugar de controlar desviaciones de un plan obsoleto, aprovechamos lo que descubrimos para mejorar el producto.

Filosofía Ágil

El objetivo primordial es entregar software de forma frecuente en un entorno cambiante. Estas metodologías se utilizan en entornos con gran variabilidad de requerimientos, involucrando al cliente en el proceso de desarrollo para conseguir una rápida retroalimentación, ya que estos encuentran imposible predecir cómo un sistema afectará sus prácticas operacionales o qué cambios habrá en el entorno (mercado o políticas de negocio) que pueden dejar el sistema completamente obsoleto.

El desarrollo ágil adopta el ciclo de vida iterativo-incremental, donde el concepto del empirismo subyace al proceso, el software no se desarrolla como una sola unidad, sino como una serie de incrementos y cada uno de ellos incluye una nueva funcionalidad del sistema. Esta es una de las diferencias con el PUD o el RUP, que, si bien utilizan un modelo de proceso iterativo - incremental, son procesos definidos.

Usualmente los incrementos se crean cada 2 o 3 semanas y se ponen a disposición del cliente. El cliente se encuentra inmerso en el proceso, ofreciendo una rápida retroalimentación que permiten una inspección y adaptación sobre los requerimientos cambiantes. Por otro lado, se reduce la cantidad de documentación con el uso de comunicaciones frecuentes e informales con el cliente en lugar de reuniones formales con documentos escritos.

Los métodos ágiles son adaptables en lugar de predictivos y están orientados a la gente en lugar del proceso.

Nace buscando un equilibrio entre hacer Software siguiendo rigurosamente un proceso donde a la larga termina siendo más importante cumplir con el proceso que entregar software de calidad y, yéndonos al otro extremo, hacer software a la ligera.

La filosofía ágil se materializa en el manifiesto ágil. Éste está basado en que la forma de trabajo va a ser sustentada en los conceptos de los procesos empíricos, bajo 3 pilares fundamentales, la transparencia, la inspección y la adaptación.

Pilares del empirismo

TRANSPARENCIA: la transparencia nos permite crecer como equipos y transformar el conocimiento que cada miembro del equipo tiene implícito en conocimiento explícito, es decir, que pase a formar parte del conocimiento de todo el

equipo. Esto ayuda a que los procesos empíricos puedan fluir y generar esa experiencia necesaria para que el equipo evolucione.

Se enfoca en “no es mi código, es nuestro código”. Todo lo que desarrollemos y nos pase dentro del equipo, pasa a ser parte de todos.

Los artefactos que tienen poca transparencia pueden conducir a decisiones que disminuyen el valor y aumentan el riesgo.

La transparencia permite la inspección. La inspección sin transparencia genera engaños y desperdicios.

INSPECCIÓN: La inspección puede realizarse para el producto, procesos, aspectos de personas, prácticas y mejoras continuas. Se realiza por todo el equipo y permite detectar varianzas o problemas potencialmente indeseables, se debe inspeccionar el progreso hacia los objetivos acordados.

Es el momento de aprender cosas nuevas que van surgiendo del trabajo que el equipo va realizando.

La inspección permite la adaptación. La inspección sin la adaptación se considera inútil.

ADAPTACIÓN: Se trata de la mejora continua, es la capacidad de adaptarse en función de los resultados de la inspección. Nos permite hacer ajustes en aspectos del proceso que se desvían fuera de los límites o si el producto es inaceptable.

Los ajustes deben realizarse lo antes posibles para minimizar la desviación normal. El equipo debe adaptarse en el momento que aprenda algo nuevo por medio de la inspección.

¿Cuándo es aplicable ágil?

Primero es importante mencionar que ágil no es una metodología o un proceso, ni tampoco un framework completamente definido, sino que es una ideología con un objetivo definido de valores y principios que guían el desarrollo del producto. Esto se debe a que definen prácticas de cómo hay que organizarse, es decir, cómo debe organizarse el equipo.

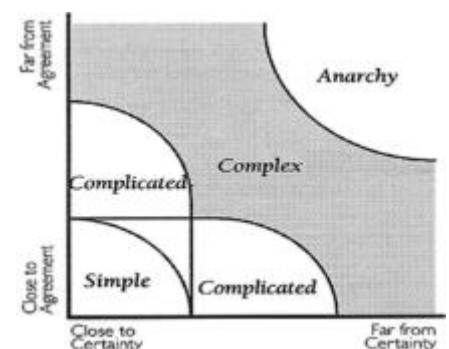
Los frameworks ágiles son distintos marcos de trabajo para implementar los principios y valores ágiles y generalmente son orientados a la gestión de proyectos y no al desarrollo.

La mayoría de los procesos ágiles son prácticas de gestión, no de ingeniería.

En función del gráfico de áreas acerca del entendimiento y certeza que se tiene sobre los problemas, decimos que las metodologías ágiles son aplicables en el contexto de lo complejo (dan buenos resultados), en el cual se ubican los nuevos productos sobre los cuales el entendimiento y la certeza que se tiene es parcial.

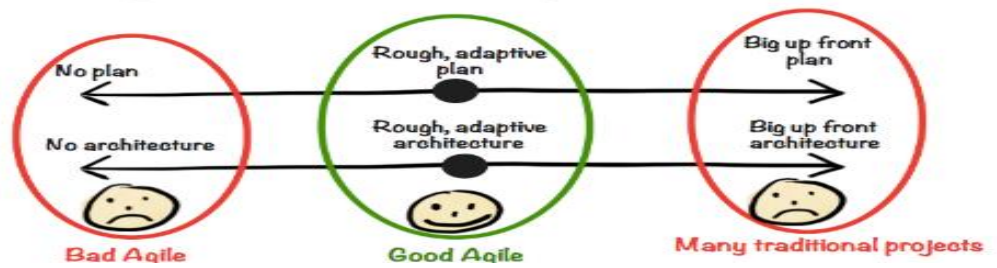
La investigación actúa en el campo de la “anarquía”, donde existe una gran incertidumbre.

Usualmente y no es una regla general, el mantenimiento se encuentra en el campo de lo relativamente simple.



Don't go overboard with Agile!

Ser ágil no es ser indisciplinado, no hay que recaer en lo que la imagen representa.



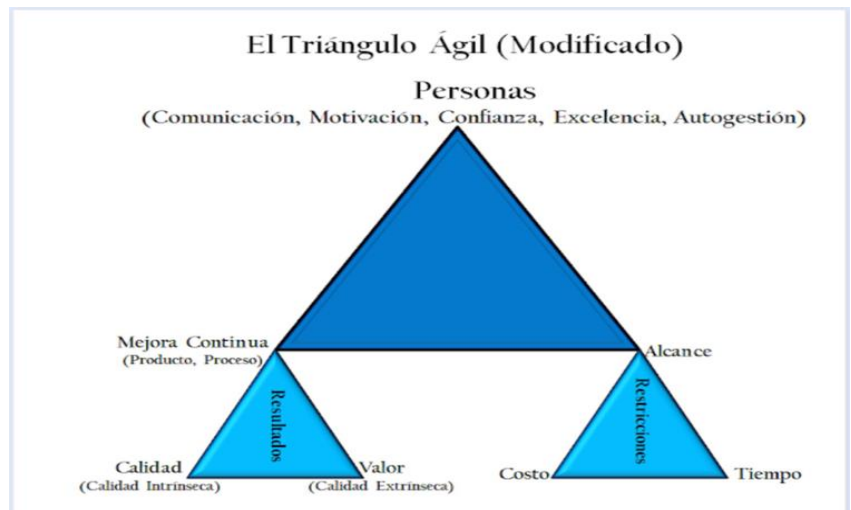
¿Por qué ir a ágil?

- Acelera entregas de productos de **valor para el cliente**.
- Permite adaptarse fácilmente a cambios
- Permite alinear los objetivos organizacionales y de TI.
- Reduce riesgos de proyecto.
- Permite una mejor visibilidad del proyecto.
- Mejora las relaciones en el equipo.
- Reduce costos de proyecto.

El triángulo ágil

El triángulo ágil apunta a que lo que realmente tiene valor que es el software funcionando para el cliente, por lo que se centra más en la calidad de producto, es decir, en la calidad intrínseca (las características de calidad que nosotros le ponemos al producto para que el producto pueda satisfacer al cliente) y la calidad extrínseca (el valor que el producto tiene para el cliente).

El triángulo ágil (modificado) apunta a un producto de software que pone a las personas (que tienen ciertas características como confianza, motivación, etc.) en el lugar más importante ya que las personas son las que hacen el software. La intención de mejora continua que tienen las personas en el proyecto del producto y del proceso.



- **Las personas son lo más importante**, donde se prioriza que haya comunicación, motivación, confianza, excelencia y autogestión. Ya que son éstas las que construyen el software.
- **Debajo encontramos** el pie, en el cual tenemos **la mejora continua** (tanto del producto con las iteraciones como del proceso, cuando habla el equipo entre sí). Es el triángulo de resultados, teniendo como pie a la calidad (la cual es calidad intrínseca, ésta representa a las características que las personas le ponen al producto para que pueda satisfacer al cliente) y al valor (calidad extrínseca, que es el calor que el producto tiene para el cliente).
- **Del otro lado** de la base podemos encontrar **el alcance**, el cual nos da cuáles serían las restricciones de nuestro producto, y encontramos el costo y el tiempo, los cuales suelen ser grandes limitantes a la hora de desarrollar software.

Finalmente se tienen en cuenta las variables que se usan para gestionar el trabajo (alcance- costo-tiempo).

Debemos hacer foco en entregarle al cliente un producto de calidad que se vaya mejorando y evolucionando en forma continua.

Requerimientos en ambientes lean ágil

Introducción al Desarrollo ágil

Requerimientos en ambientes ágiles – User Stories

Lo más difícil del software es decidir que software queremos construir, ya que las cuestiones tecnológicas no son el impedimento, sino que los proyectos de software fracasan debido a que no se cumplen con las expectativas del cliente: lo que se construye no es lo que el usuario quería o necesitaba. Este problema se advierte en la crisis del software. Probablemente una de estas razones es no asimilar la complejidad que supone hacer software y que la etapa de relevar requerimientos (la cual es social y no técnica) suele ser subestimada.

También, es muy común que haya problemas de comunicación entre los clientes y el equipo. Cuando ninguno de los dos domina la comunicación, el proyecto fracasa.

Debe haber un balance: si se impone el cliente, exige plazos irreales; si se imponen los devs, el lenguaje técnico opaca el objetivo del negocio.

Lo óptimo es que haya una comunicación de ambas partes, ya que los proyectos suelen fracasar si solo una parte domina la comunicación y asignación de recursos.

Como usuarios, ver “early versions” o entregas tempranas es una ventaja ya que permite que nos surjan nuevas ideas o que nos demos cuenta de que no es exactamente lo que queríamos antes de que el proyecto siga avanzando por un camino equivocado.

Es por esto que no es posible realizar un diagrama PERT perfecto mostrando todo lo que se debe realizar en un proyecto (no se conocen de arranque todas las actividades).

Fundamentación de los requerimientos ágiles

El fondo de la cuestión está en los principios y valores explicitados en el manifiesto ágil, fundamentalmente la entrega frecuente y temprana de software valioso y la posibilidad de recibir cambios de requerimientos aun en etapas finales. Esto último implica una predisposición, una actitud del equipo frente a la posibilidad de cambios. La base de los requerimientos en ambientes ágiles tiene que ver con la posibilidad de comenzar a construir el producto cuando el producto no está completamente definido.

Los requerimientos ágiles se basan en 3 pilares.

- **Usar el valor para construir el producto correcto:** Lo único importante es dejar contento al cliente con software funcionando que le sirva, por lo que, el foco de este enfoque de gestión ágil apunta a construir valor de negocio. Es decir, el software que creamos es para ayudar a alguien más a cumplir con sus objetivos y trabajo, por eso decimos que debemos enfocarnos en construir el producto correcto que le dé valor al negocio. Hay que construir un producto que le haga la vida más fácil a las personas que lo utilizan.
- **Determinar que es “sólo lo suficiente”:** hace referencia a que los requerimientos deben ser encontrados de a poco, no buscar todos los requerimientos desde un inicio y quedarnos solo con eso. Este pilar postula que se tiene que empezar con lo mínimo necesario para generar realimentación y alimentar el empirismo y a partir de allí, se avanza continuamente.
- **Usar historias y modelos para mostrar que construir:** la parte de desarrollo de requerimientos está muy enfocada a la idea de construir junto con el cliente, respetando el principio ágil de “técnicos y no técnicos trabajando juntos”. Se busca trabajar con un referente que esté del lado del negocio, que tenga el conocimiento del producto y que trabaje junto con el equipo.

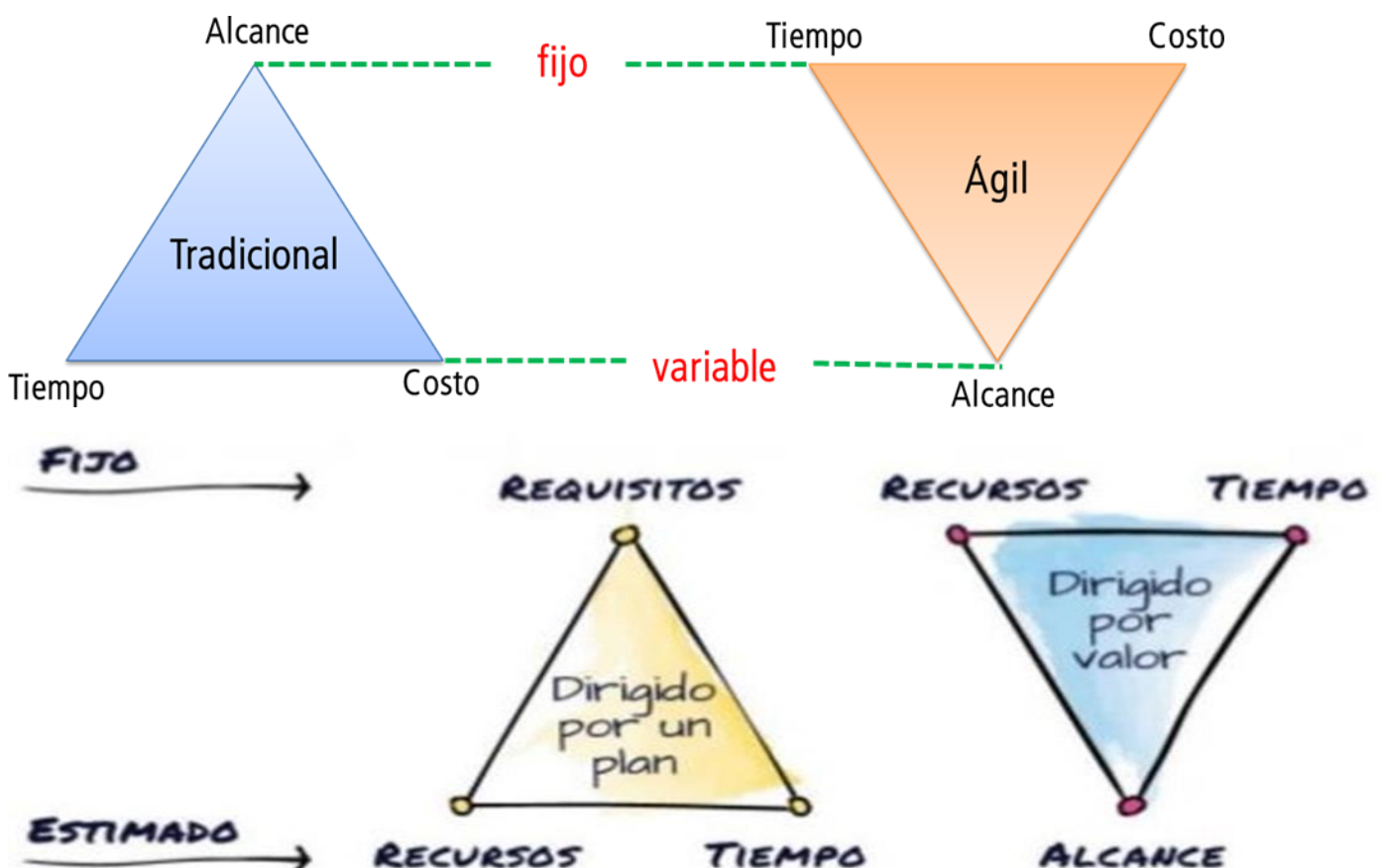
La etapa de capturar los requerimientos se transforma en una etapa de trabajo muy integrado entre el cliente y nosotros. Por eso el mejor medio de comunicación es la comunicación cara a cara porque **la idea es armar equipos de trabajo en los que con técnicas de descubrimiento (las User Stories) se pueda lograr crear la visión de qué producto se quiere construir y a partir de ahí, avanzar.**

El enfoque de requerimientos ágiles debe estar adherido al manifiesto ágil, es decir, a cumplir con los valores y con los principios que el manifiesto plantea.

Recordemos que la ingeniería de requerimientos tiene 2 procesos: por un lado, **el desarrollo de requerimientos** y por otro lado **la gestión de los requerimientos**. La gestión está más vinculada con mantener los requerimientos íntegros, bien identificados, bien trazados y que si cambian los requerimientos se actualicen todos los lugares donde se tengan que actualizar esa definición de requerimientos. Esto ocurre todo el tiempo mientras estamos trabajando con un producto de software. La parte de desarrollo de requerimientos, que es la parte que tiene que ver con identificar, especificar y validar los requerimientos.

La triple restricción – enfoque ágil

En el enfoque ágil, **se trata de aceptar la realidad en la cual el alcance (requerimientos) del producto puede cambiar a lo largo del desarrollo, y no es posible definirlos al comienzo de este.** Entonces, **el enfoque trata de dejar el tiempo y los costos fijos, teniendo en cuenta “cuánto software” le puedo entregar en tanto tiempo y con tal equipo de desarrollo.**



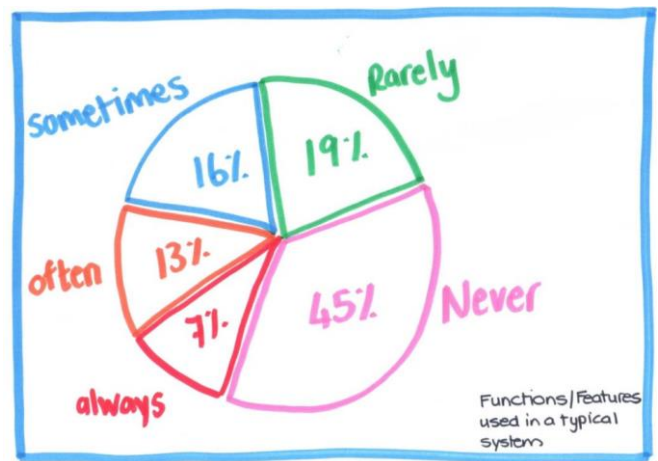
Por otro lado, en el enfoque tradicional, el alcance se fija y en función de eso derivan los recursos (costo) y el tiempo. El cliente desea controlar el alcance, ya que este comprende lo que él realmente necesita. Luego el cliente pregunta por el tiempo y el costo, los cuales son derivados por el equipo. Por esta razón es tan costosa incorporar cambios de requerimientos, ya que se estaría modificando el alcance y como consecuencia los recursos y el tiempo.

BRUF – Big Requirements Up Front

La imagen adjuntada expresa la frecuencia con la que los usuarios utilizan determinadas funcionalidades de un software. A lo que se apunta con este concepto de BRUF es a notar que, en una primera instancia de un proyecto de software, desarrollar sólo unas pocas funcionalidades de gran valor e importancia para el cliente, puede cubrir gran parte de sus necesidades, a medida que se desarrolla el resto de los requerimientos.

Además, es prácticamente imposible conocer al 100% todas las necesidades de los usuarios, ya que ni ellos saben concretamente qué necesitan, ni cómo explicar de forma abarcadora y completa sus necesidades. Por otro lado, esas

necesidades que tienen los usuarios al momento de ser relevadas, con el tiempo probablemente irán cambiando, con lo cual, la priorización es un factor fundamental en esa etapa, donde el equipo en conjunto con el Product Owner, deberán ser capaces de decidir cuáles son aquellos requerimientos indispensables para el negocio, de manera que, si se implementan esas necesidades, se puede entregar una nueva versión del producto que sea de valor para el cliente.



En contraposición con el enfoque tradicional, donde teníamos un cliente que no estaba dispuesto a comprometerse y dejaba la priorización de los requerimientos al equipo que hace el software, se terminaba construyendo un producto que no dejaba satisfecho al cliente. En ágil es el cliente el encargado de esta priorización, lo que lleva a que se construya el producto deseado.

Con esta situación se permite la compensación de no tener documentos de requerimientos formales escritos, detallando todos los puntos.

Requerimientos Just in Time

La gestión ágil de requerimientos sostiene los **requerimientos justo a tiempo**, asociados a los principios de eliminar el desperdicio y diferir compromisos, dentro de la filosofía Lean.

Se empieza con una visión del producto y con algunos requerimientos identificados. A partir de esta base, se comienza a construir el producto y se incorpora al cliente para obtener retroalimentación. **El Product Backlog nunca está al 100%**

En el enfoque tradicional se aplica el "Up Front Specification", mediante el cual se especifica todos los requerimientos antes de comenzar el desarrollo. Esto implica que un error en la captura de requerimientos tiene un alto impacto. En cambio, los requerimientos en ambientes ágiles al ser consensuado continuamente con el cliente, el impacto que se tiene es menor.

El producto no va a estar especificado al 100% desde el principio ni muchísimo menos, sino que se van a ir encontrando requerimientos y describiendo los requerimientos que hacen falta conforme haga falta. “ni antes, ni después”.

Fuente de requerimientos

Los requerimientos ágiles, basados en el principio de que el mejor medio de comunicación es el cara a cara, promueven que los requerimientos del software se obtienen conversando cara a cara con el cliente.

En los ambientes tradicionales, los requerimientos se encuentran especificados en un documento de especificación de requerimientos (ERS) y el cliente luego lo lee para validarlo (sería lo que en el gráfico dice “papel”).

Los agilistas no escriben una ERS, pero lo compensan con la disponibilidad del cliente para resolver las dudas. Esto permite reducir la cantidad de documentación formal. La elicitación de requerimientos es un proceso que requiere de conversar de forma personal, analizando lo que los clientes dicen y los gestos que manifiestan.

El desarrollo ágil no implica la no documentación o la no especificación de requerimientos, sino que prioriza documentar y especificar aquellos que sean de valor para el cliente, antes que hacer toda una ERS errónea.

El cara-a-cara permite que fluya información vocal, subvocal, gestual con realimentación rápida.



Momento de la captura de requerimientos

En el enfoque tradicional, los requerimientos son definidos al principio del proyecto. El caso particular del PUD, que es un proceso definido con un modelo de proceso iterativo e incremental, el flujo de trabajo de requerimientos tiene un gran peso en la fase inicial y de elaboración.

En el enfoque ágil, los requerimientos son relevados continuamente durante todo el proyecto. El inicio de la construcción del software se da en un contexto en el cual los requerimientos no están completamente definidos, sin embargo, se tiene una visión.

Contenedor y persistencia de los requerimientos

El enfoque tradicional, la ERS contiene los requerimientos a lo largo del ciclo de vida del producto todo el tiempo. En caso de que surjan cambios, es necesario actualizar la ERS. En definitiva, es el documento de más alto nivel que define las características del sistema.

En el enfoque ágil, se utiliza al Product Backlog como un contenedor transitorio de los requerimientos, ya que una vez implementados estos se “persisten” en el producto funcionando. El detalle de valor queda persistente en los casos de prueba que se construyen en el Testing.

Tipos de requerimientos

La parte baja de la pirámide, corresponde a los requerimientos del Dominio de la solución (esto no le interesa al cliente, es cómo el equipo transforma sus necesidades en código ejecutable). Lo que está por encima, es decir, Requerimientos de Usuario y Requerimientos de Negocio, corresponden al Dominio del Problema, y es lo que al cliente sí le interesa.

A nivel de la gestión ágil de requerimientos se trabaja con:

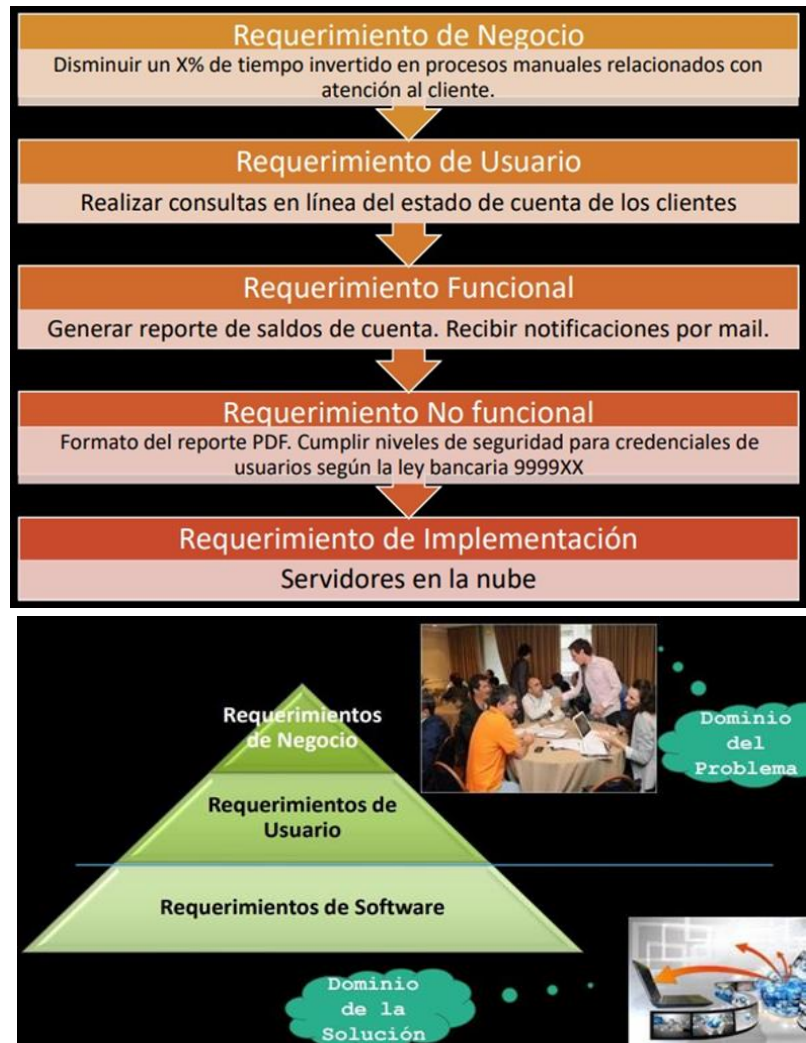
- Requerimiento de negocio
- Requerimiento de usuario

Las User Stories son requerimientos de usuario alineados a un requerimiento de negocio.

El valor surge de cumplir con los objetivos del negocio, pero son los requerimientos de usuario los que definen las acciones concretas para alcanzarlo

Las User Stories no sirven para especificar requerimientos de software, sirven para identificar requerimientos de usuario.

Dado que el enfoque ágil se centra en resolver necesidades reales del usuario, el Product Owner es responsable de priorizar e identificar las US que generen el mayor impacto tanto para el cliente como para el negocio.



¿Qué debemos asumir?

- Los cambios son la única constante. Todo el tiempo va a haber cambios en los requerimientos.
- Stakeholders: no son todos los que están. Debemos saber quiénes son los involucrados y los involucrados son todos los que tiene algo que decir respecto del producto. El PO es el representante de todas estas personas y el que se comunica con todas ellas para luego ser él mismo el que se comunica con el equipo que hace el software.
- Siempre se cumple eso de que: “El usuario dice lo que quiere cuando recibe lo que pidió”.
- No hay técnicas ni herramientas que sirvan para todos los casos. Debemos detectar para cada situación cuál es la herramienta más adecuada.
- Lo importante no es entregar una salida, un requerimiento, lo importante es entregar, un resultado, una solución de “valor”.

User Stories

Son una técnica para trabajar requerimientos de usuario en ambientes ágiles. La US contiene una descripción corta de una funcionalidad que se espera del producto valorada por un usuario del sistema.

Es un recordatorio visible: posterga la conversación detallada entre el Product Owner y el equipo hasta el momento en que realmente se va a desarrollar la funcionalidad.

Es un requerimiento a nivel de usuario, no al nivel de sistema, por lo que se encuentra a un nivel de abstracción más elevado. Es un ítem de planificación que no acompaña al producto durante todo el ciclo de vida. Para eso está el manual de usuario. Las US son escritas por el Product Owner.

Se dice que son multipropósito, debido a que cumplen diferentes funciones en el desarrollo de un software:

- Representan una necesidad de usuario, de forma no detallada.
- Conforman una descripción del producto.
- Son utilizadas como ítems de planificación.
- Se utilizan como recordatorios de futuras conversaciones acerca del producto que se desea construir.
- Junto con el código ejecutable, permiten documentar el producto.

Esta técnica nace por la dificultad de explicarle a todas las partes interesadas qué va a hacer el sistema, ya que cada persona tiene intereses y expectativas distintas.

Las Historias de Usuario postergan las decisiones para tomarlas a lo largo del desarrollo: a medida que el proyecto avanza se sabe mas y se puede entregar valor.

Partes de la User Story

Tarjeta (Card)

Breve descripción la necesidad, utilizada como recordatorio para planificar y postergar los detalles hasta el momento adecuado.

Conversación (Conversation)

Representa el diálogo continuo entre el PO y el equipo para definir los detalles que la tarjeta no incluye. Aplica principios ágiles donde se prioriza colaborar entre perfiles técnicos y de negocio sobre contratos rígidos. Su resultado se refleja directamente en la tarjeta y en las pruebas de aceptación.

Confirmación (Confirmation)

Criterios y pruebas de aceptación acordados entre el Product Owner y el equipo para verificar que la funcionalidad realmente se implementó. El Product Owner es el responsable final de validar estas pruebas para declarar la historia como completada.

Ventajas

- Prioriza la comunicación verbal por sobre la escrita (recordar la conversación es la parte más importante).
- Son comprensibles tanto para el cliente/usuario (Product Owner) como para el equipo de desarrollo (recordar que se escriben en términos de negocio).
- Tienen el tamaño justo para permitir planificar.

- Permiten el desarrollo iterativo al dividir todas las funcionalidades del producto en US.
- Permiten desestimar los detalles, hasta que se logre una mejor comprensión sobre cuál es la necesidad concreta de los usuarios.

Desventajas

- En proyectos grandes, es difícil entender la relación entre U.S (son independientes entre sí).
- No son apropiadas para equipos demasiado grandes (por el tema de priorizar la conversación verbal, por sobre la escrita).

Product Backlog y las User Stories

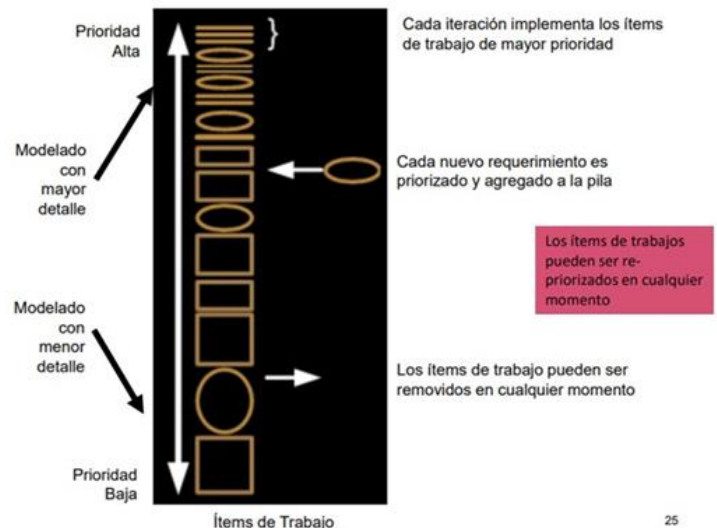
Es una pila de PBIs (Product Backlog Items) con sus correspondientes estimaciones de tamaño, en un formato determinado. En la parte más alta de la pila, se ubican las U.S. de mayor prioridad (que son aquellas U.S. con granularidad fina), y las de menor prioridad (normalmente aquellas User de granularidad gruesa, sobre las cuales no se tiene mucho detalle ya que no se consideran indispensables) al fondo. En cualquier momento se puede cambiar esa priorización, según cómo lo indique el Product Owner, quien es el encargado de realizar el proceso de priorización.

En cualquier momento del proyecto se van agregando nuevas U.S. y actividades a realizar, según se vayan detectando nuevos requerimientos.

Para quitar una U.S. del Product Backlog, se planifica una iteración y se decide, según la velocidad de trabajo del equipo [Story Points / iteración], cuántas U.S. de la pila (las de mayor prioridad que están arriba) se quitarán para implementarlas en la próxima iteración.

Debido a estas dos situaciones (agregar y quitar ítems), **nunca** encontraremos el 100% de los requerimientos de usuario en el Product Backlog.

El Product Owner
Prioriza las
historias
en el
Product
Backlog



Porciones verticales

Las US son porciones verticales, es decir, para poder agregar valor al cliente debemos tener un poco de interfaz, un poco de lógica de negocio y otro poco de base de datos.

Debemos diseñar el pedacito de cada cosa que nos hace falta para que una determinada característica funcione.

Story 1	Story 2
GUI	
Business Logic	
Database	

Tamaño

Es recomendable tener una gran cantidad de historias con tamaños relativamente pequeños, que tener historias muy grandes.

El tamaño de las U.S. se mide en Story Points.

Hay 3 aspectos que determinan el tamaño de una U.S.:

- **Complejidad:** hace referencia a una dificultad inherente a la funcionalidad, que no permiten dividirla en U.S. más pequeñas.
- **Esfuerzo:** es el tiempo en horas de trabajo que requerirá la implementación.
- **Incertidumbre**
 - Técnica: cuando se tiene dudas en el dominio de la solución, por ejemplo, el uso de una cierta tecnología no muy conocida por el equipo.
 - De negocio: cuando hay incertidumbre respecto a cómo interactuará el usuario con el sistema, lo cual es causado por falta de conocimiento del dominio del negocio.

Niveles de abstracción

1. **User Story:** cumple con el criterio de Listo (DoR) para ingresar a una iteración.

2. Épica

Es una US demasiado grande para completarse dentro de una sola iteración (las US deben empezar y terminar en el mismo ciclo).

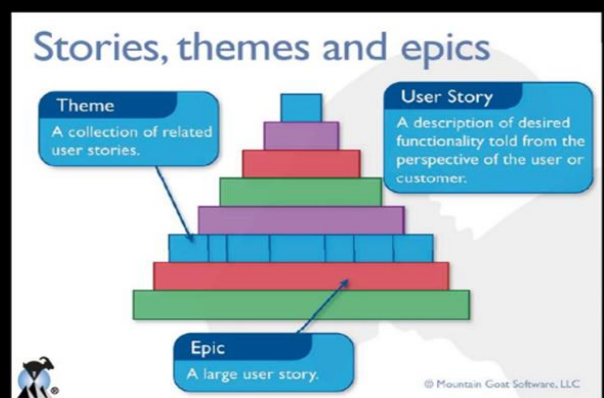
Comparte la misma sintaxis que una User Story común y se identifica cuando no cumple con el modelo de calidad INVEST.

Sirve para registrar requerimientos que no son prioritarios al inicio, donde alcanza con tener una idea general. Cuando llega el momento de construirlos, la épica se divide en historias más chicas para poder planificarlas y desarrollarlas en iteraciones sucesivas.

3. **Temas:** son un conjunto de User Stories agrupadas, debido a que conceptualmente están relacionadas. Se las agrupa para tratarlas como una entidad simple a la hora de estimar o planificar.

Ambos (Épicas y Temas) permiten reducir los esfuerzos del equipo a la hora de estimar.

Diferentes niveles de abstracción



Modelado de Roles

Se utilizan las **tarjetas de rol de usuario**, aquí se anotan las características en general que va a tener el usuario.

El tema crítico de la funcionalidad está relacionado con quien va a utilizar dicha funcionalidad. Por esta razón existe una tendencia por ocuparse de quien va a usar el sistema, especificando los roles de usuario donde definimos las características del usuario respecto al producto y en relación con su perfil genérico.

Técnicas adicionales

1. **Personas:** esta técnica busca ser más específica acerca de la descripción de la persona concreta que va a utilizar el sistema. Este análisis es exhaustivo y por lo tanto complejo. Si los roles son clases, las personas serían los objetos.
2. **Personajes extremos:** que personas en términos extremos podrían llegar a utilizar el software.
3. **Proxies (Usuarios Representantes):** es un representante de todos los usuarios. Debe tener capacidad de tomar decisiones desde el negocio.

Criterios de aceptación

Es una nota no obligatoria que define un acuerdo respecto a cómo se debe comportar el software para que el PO acepte la implementación, es recomendable que este debido a que sino los desarrolladores no tendrían un parámetro para definir si la misma fue cumplimentada o no. Lo fundamental es que esté escrito en términos objetivos y concretos.

El criterio de aceptación es información concreta que tiene que servirnos a nosotros para saber si lo que implementamos es correcto o no. Debemos saber si el PO nos va a aceptar esta característica implementada o no, porque la va a aceptar si respeta los criterios de aceptación.

- Definen límites para una User Story (US)
- Ayudan a que los PO respondan lo que necesitan para que la US provea valor (requerimientos funcionales mínimos).
- Ayudan a que el equipo tenga una visión compartida de la US.
- Ayudan a desarrolladores y testers a derivar las pruebas.
- Ayudan a los desarrolladores a saber cuándo parar de agregar funcionalidad en una US.

¿Cuáles son los criterios de aceptación buenos?

- **Definen una intención**, no una solución. Ejemplo: El usuario debe elegir al menos una cuenta para operar.
- **Son independientes de la implementación.**
- **Relativamente de alto nivel, no es necesario que se escriba cada detalle.**

¿Y los detalles, a dónde van?

Detalles como:

- El encabezado de la columna se nombra "Saldo"
- El formato del saldo es 999.999.999,99
- Debería usarse una lista desplegable en lugar de un Checkbox.

Estos detalles que son el resultado de las conversaciones con el PO y el equipo puede capturarlos en dos lugares:

- **Documentación interna de los equipos**

- **Pruebas de aceptación automatizadas.**

Cada equipo decide dónde guardar el detalle y cómo lo va a mantener.

Pruebas de aceptación

Son declaraciones de intención de que hay que probar. El conocimiento del negocio está inmerso en las pruebas de aceptación. Se prueban escenarios exitosos y escenarios que fallen. En las pruebas de aceptación se encuentran las más importantes, ya que **no es posible escribir todas las posibles pruebas de aceptación**.

El Product Owner acepta la User como implementada si todas las pruebas de aceptación se cumplen.

Expresan detalles resultantes de las conversaciones entre clientes y desarrolladores; **suelen usarse para completar detalles de la US**. Es importante mencionar que las mismas **deben escribirse antes que la programación empiece**, y que **se recomienda que las escriba el cliente**.

DoR (Definition of Ready) – Definición de Listo

Es una medida de calidad que **determina si la User Story está en condiciones de entrar a una iteración de desarrollo**. Para que la US pueda ser implementada, **mínimamente debe satisfacer el INVEST Model**. Entre el equipo de desarrollo y el PO pueden definir características o condiciones extras al INVEST para que una US se considere lista para entrar a la iteración de desarrollo.

INVEST Model

Es un modelo de calidad que nos ayuda a verificar si la User cuenta con las características de la calidad mínima para satisfacer la definición de Ready y poder ingresar en una iteración de desarrollo.

- **Independiente:** la User es calendarizable e implementable en cualquier orden. Esto implica que no existe una dependencia con otra User para desarrollarla o para mostrarla al Product Owner. El PO tiene la libertad de priorizarlas en el Product Backlog como él desee.
- **Negociable:** Las US no son contratos estrictos, sino más bien pequeñas descripciones de funcionalidades donde se describen las necesidades de los usuarios, y los detalles se dejan para negociar en conversaciones futuras entre clientes y el equipo de desarrollo. Esto implica que no deben detallarse todos los aspectos de forma exhaustiva, sino más bien, servir a modo de recordatorio para negociar aquellos aspectos en un futuro cuando se deba implementar.
El cliente debe expresar lo que necesita, es decir que está centrada en el “qué” no el “cómo”. El cómo es decisión del equipo y debe ser negociable con el Product Owner.
La US debe estar escrita en términos de qué necesita el usuario y no de cómo lo vamos a implementar.
- **Valuable:** Una US debe ser valorable para los clientes o usuarios (el para qué de la descripción). Es decir, las U.S. deben aportar valor de negocio a quiénes estará destinado el producto, no para el equipo de desarrollo. Esto implica dejar de lado en US, por ejemplo, las tecnologías utilizadas para el desarrollo del producto.
- **Estimable:** la US debe contener la cantidad de información suficiente para estimar el tamaño, la complejidad y el esfuerzo necesario para realizarlo. Sin poder estimarla no es posible determinar si se puede finalizar en una iteración por lo tanto no puede ser construida (No puedo saber cuánto tardaría).
Hay tres razones por las cuales una US puede no ser estimable:
 1. El equipo tiene incertidumbre acerca del dominio.
 - a. Deberán discutir con el Product Owner para esclarecer todas las dudas.
 2. El equipo tiene incertidumbre técnica, es decir, sobre el dominio de la solución.
 - a. Deberá destinarse uno o más miembros del equipo a uno o más Spikes.
 3. La US es demasiado grande.

a. En este caso, deberá dividirse en US más pequeñas.

- **Small (Pequeña):** la US debe ser lo suficientemente pequeña como para ser finalizada en una iteración. Se debe comenzar y terminar una característica en una iteración, no debe haber trabajos a medias. El tamaño es relativo. ya que depende del equipo, de la experiencia que tiene, de su capacidad de trabajo y también depende de la duración de la iteración. Ágil aplica gestión binaria, es decir, si se puede completar la US en la iteración, es pequeña. No aplica porcentaje de avances.
- **Testeable:** la US debe cumplir con las pruebas de aceptación para poder probar si la misma fue o no implementada (se muestra que está hecha y demostrable).

DoD (Definition of Done) – Definición de Hecho

Determina si la US está terminada. Son los criterios que establecen cuándo una User Story ha sido implementada de forma correcta y completa, de forma que puede ser mostrada al Product Owner, para que éste decida si se pasa a producción o no. En el momento que una U.S. cumple con todo lo pactado en esta Definición de Hecho, se considera que esa US está lista para ser desplegada y aportar valor al cliente.

Esto permite crear transparencia, debido a que proporciona a todo el equipo una comprensión exhaustiva de qué trabajo se realizó, para lograr ese incremento.

Spikes

Son un tipo especial de US que se producen por la incertidumbre que la misma presenta, la cual imposibilita que pueda ser estimada y por lo tanto no cumple con la definición de listo. Una vez resuelta la incertidumbre, la Spike se convierte en una o más US. Es una característica más del producto a la cual el equipo le tiene que dedicar tiempo para:

1. Familiarizarse con una nueva tecnología o dominio.
2. Investigar y prototipar para ganar confianza frente a:
 - a. Riesgos tecnológicos.
 - b. Riesgos funcionales, donde no está claro cómo debe reaccionar el sistema para satisfacer las necesidades del usuario.

Se usan para quitar el riesgo e incertidumbre de una US y otra faceta del proyecto. Se clasifican en técnicas y funcionales y pueden utilizarse para:

1. Inversión básica para familiarizar al equipo con una nueva tecnología o dominio.
2. Analizar el comportamiento de una historia compleja y poder dividirla en piezas manejables.
3. Ganar confianza frente a riesgos tecnológicos, investigando o prototipando.
4. Ganar confianza frente a riesgos funcionales, donde no está claro cómo el sistema debe resolver la interacción con el usuario para alcanzar el beneficio esperado.

Spikes técnicas

Sirve para evaluar tecnologías o métodos de implementación antes de construir una funcionalidad.

Reducen la incertidumbre técnica para que el equipo pueda comprometerse a fechas fijas sin sorpresas.

Su alcance y ejecución dependen por completo del equipo técnico.

Spikes funcionales

Utilizada cuando hay cierta incertidumbre respecto de cómo el usuario interactúa con el sistema, usualmente son mejor evaluadas con prototipos para obtener retroalimentación de los usuarios involucrados.

Dependen de definiciones del negocio, del PO.

Lineamientos para Spikes

Medibles y con fin claro: Una spike se tiene que poder estimar, contar con criterios de aceptación y arrojar un resultado.

Uso como último recurso: El spike no se usa para cualquier duda cotidiana, sino cuando tenemos incertidumbre o dudas técnicas o funcionales que no nos dejan avanzar.

Separar la investigación del desarrollo: Una Spike y sus US derivadas no se deben resolver en la misma iteración. Solo si la Spike se resuelve rápido su incertidumbre y la implementación de la US es rápida.



DIFERIR EL ANÁLISIS
DETALLADO TAN TARDE
COMO SEA POSIBLE, LO QUE
ES JUSTO ANTES DE QUE EL
TRABAJO COMIENCE.



HASTA ENTONCES, SE
CAPTURAN
REQUERIMIENTOS EN LA
FORMA DE "USER STORIES".



LAS USER STORIES NO SON
REQUERIMIENTOS DE
SOFTWARE, NO NECESITAN
SER DESCRIPCIONES
EXHAUSTIVAS DE LA
FUNCIONALIDAD DEL
SISTEMA.

Investment Themes (Temas de Inversión)

Iniciativas estratégicas que orientan el presupuesto de la empresa en sistemas, productos y servicios para ventajas competitivas.

Son inversiones en cinco áreas:

Productos actuales: Mejoras continuas, soporte y mantenimiento de lo existente.

Nuevos productos a corto plazo: Soluciones para aumentar ganancias de inmediato.

Innovación a futuro: Investigación y desarrollo de productos o servicios avanzados.

Apuestas a largo plazo: Proyectos estratégicos de hoy que darán beneficios recién dentro de varios años.

Estrategias de retiro: Reducción y desmantelamiento de productos que llegan al final de su vida útil.

Estimación en ambientes ágiles

Estimar es el **proceso de obtener una aproximación sobre una medida con el objetivo de predecir la completitud y gestionar los riesgos en el contexto de un proyecto**. Se relaciona con los objetivos del negocio, compromisos y control.

Según McConell **una estimación en el contexto del software es una predicción de cuánto tiempo durará o costará un proyecto**. Es considerada una de las actividades más complejas en el desarrollo de software.

Las estimaciones no son planes ni compromisos, sino que **sirven de base para planificar, pero el plan final no tiene por qué coincidir con lo estimado**.

Errores en las estimaciones

1. Información imprecisa acerca del software a estimar o acerca de la capacidad para realizar el proyecto.
2. Demasiado caos en el proyecto (mal definido el proyecto).
3. Imprecisión generada por el proceso de estimación.
4. Una de las fuentes más común es omitir actividades necesarias para la estimación del proyecto tales como, requerimientos faltantes, licencias, reuniones, revisiones, etc.
5. Se olvidan del detalle que planear no es estimar y estimar no es planear, sino que las estimaciones son la base de los planes, pero los planes no tienen que ser lo mismo que lo estimado.
6. A mayor diferencia entre lo planeado y lo estimado, mayor el riesgo.
7. Las estimaciones no son compromisos.

¿Por qué estimamos?

Existen dos razones principales por las cuales se llevan adelante las estimaciones:

- Para predecir completitud.
- Para administrar riesgos.

Antes de que el proyecto comience, el líder del proyecto y el equipo de software deben estimar el **trabajo** que habrá de realizarse, los **recursos** que se requerirán y el **tiempo** que transcurrirá desde el principio hasta el final. Las estimaciones requieren:

- **Experiencia**.
- **Acceso a buena información histórica (métricas)**.
- **El valor para comprometerse con predicciones cuantitativas cuando la información cualitativa es todo lo que existe**.

Siempre se desea saber muy al inicio del proyecto cuál será el costo, el tiempo que llevará, entre otros aspectos y como aún no se conoce con precisión cuál será el alcance del producto resulta complejo predecir con exactitud estos aspectos, es por ello que **a medida que avanza el proyecto, las estimaciones son más certeras, dado que se cuenta con**

mayor información sobre la cual realizar las estimaciones. (Es por ello que se suele decir que típicamente la primera estimación difiere hasta un 400%).

Al inicio del desarrollo hay una estimación errónea de 2 a 4 veces más, esto se debe a dos aspectos:

1. Al inicio del proyecto tenemos poca información y demasiada incertidumbre, lo que aumenta el riesgo de generar estimaciones erróneas.
2. Las estimaciones son de naturaleza optimista, por eso las estimaciones deben ser recalculadas conforme se avanza con el proyecto.
3. Se suele decir: “el que no sabe estimar, estima muchas veces, y el que no sabe planificar, planifica muchas veces”.

Proceso de estimación

Para ver cómo y qué se estima en el ambiente tradicional ver el punto de estimación en el título “[Plan de proyecto](#)” en la unidad 1 de este apunte.

Métodos utilizados para estimar

Se recomienda utilizar diferentes métodos de estimación y luego contrastar (comparar). Todos tienen un “Factor de Ajuste” que permite que el método cierre (calibraciones).

1. Basados en la experiencia

- a. **Datos históricos:** consiste en recolectar datos básicos de otros proyectos para ir generando una base de conocimientos que sean de utilidad para futuras estimaciones, con esto se busca una alternativa en la cual el conocimiento de cada individuo se transfiere a la organización y no estamos atados a una única persona. Se deben utilizar para no tener problemas entre desarrolladores y clientes. Esto no va acorde a la gestión ágil, que dice que la experiencia no se puede extrapolar a otros equipos.
 - i. **Industry Data:** datos de otras organizaciones que aportan al mercado y desarrollan productos con algún grado de semejanza y que permite una comparación básica.
 - ii. **Historical Data:** datos de la organización de proyectos que se desarrollaron y ya se cerraron.
 - iii. **Project Data:** datos del proyecto, pero de etapas anteriores a la que se está estimando.
- b. **Juicio experto:** es uno de los más utilizados, el problema se presenta cuando se utiliza como única técnica de estimación el juicio experto puro. Existen dos técnicas para aplicarlo:
 - i. **Juicio experto puro:** un experto estudia las especificaciones y realiza una estimación. Esta estimación se basa fundamentalmente en la experiencia del experto. (el problema es que, si el experto se va de la organización, esta pierde su capacidad de estimación). Se puede utilizar el “método de optimista, pesimista, habitual”, en el que se le pregunta al experto sobre 3 valores distintos y se aplica una fórmula para calcular la estimación.
 - ii. **Juicio Wideband Delphi:** es similar al anterior pero grupal, un grupo de personas se reúne con el objetivo de converger a una estimación común, tanto en esfuerzo como en duración. Hay un coordinador que va a reunir las estimaciones individuales de cada uno de los expertos cuando les dieron las especificaciones, se opina sobre las estimaciones ajenas de manera anónima, se vuelve a discutir, revisan, las vuelven a enviar al coordinador y se repite hasta que todos estén de acuerdo de forma razonable. Este método es tomado como base para el Poker Planning.

2. Analogía

Se comparan los factores a estimar de forma relativa con respecto a las estimaciones de otros proyectos similares. Es un método que tiene mucho error dado que el conocimiento en un proyecto no es extrapolable a otro. Es decir, consiste en tomar a un proyecto e ir comparando factores, por ejemplo:

- Tamaño: es menor o mayor al proyecto anterior.

- Complejidad: más complejo de lo usual.
- Usuarios: si hay más usuarios habrá más complicaciones, como la concurrencia o los datos a almacenar.

3. Otros

- Basados exclusivamente en los recursos:** consiste en analizar el personal que se tiene y de cuánto tiempo se dispone del mismo, haciendo las estimaciones exclusivamente en función de este factor. En la realización: “El trabajo se expande hasta consumir todos los recursos disponibles, independientemente si se alcanzan o no los objetivos.”
- Basados en la capacidad del cliente o exclusivamente en el mercado:** ponen el foco en el cliente y realizan las estimaciones en función de su capacidad de pago. En este método no se evalúa al producto que se está intentando vender, sino a quien se lo vende. A veces se cobra menos para que nos elijan antes que a la competencia.
- Basados en los componentes del producto o en el proceso de desarrollo:** se pueden aplicar 2 enfoques:
 - Bottom-up:** se descompone el producto en unidades lo más pequeñas posible. Se estima cada unidad y luego se hace la estimación total.
 - Top-down:** se descompone el producto en grandes bloques y luego se estima cada uno de los componentes.
- Métodos algorítmicos:** se ejecutan algoritmos que tienen como parámetros de entrada medidas cuantitativas de tamaño, complejidad y conocimiento de dominio. Cada variable tendrá asociada un factor multiplicativo y en función de esto se obtiene como resultado un valor de esfuerzo. Es importante destacar que, a pesar de ser un cálculo matemático, puede no ser certero igual.

Problemas de estimaciones tradicionales

- **Estimar demasiado pronto.**
- **Resistencia a las reestimaciones.**
- Estimaciones que **buscan precisión.**
- Estimaciones **hechas por gente distinta a la que hace el trabajo.**
- Estimaciones que **pretenden ser absolutas** (inflexibles).

Estimaciones en ambientes ágiles

Son aproximacio. numéricas asociadas a una probabilidad de que cierto evento ocurra. La filosofía ágil plantea que quien estima, es quien debe hacer el trabajo, con lo cual es el equipo de desarrollo quien realiza las estimaciones, de forma colectiva (**menos el Product Owner, no participa en estimaciones**). Al ser de esta manera, la estimación se ajusta mejor, debido a que se generan debates entre los miembros del equipo, lo cual puede abrir puntos de vista no detectados por otros, haciendo que ese proceso de estimar se vuelva lo más **preciso** posible (a diferencia de las metodologías tradicionales, donde se estima individualmente y se depende sólo de ese miembro).

Para estimar de esta forma, **es fundamental que los miembros del equipo tengan una actitud en escucha**, donde **todos puedan opinar libremente** y no haya influencias que “tapen” la opinión de otros.

Frente a la posibilidad de estimar mal y generar enojo por parte del cliente, tenemos dos reacciones posibles:

Como la estimación es un trabajo a riesgo, **se subestima**, o sea, simplificando sin considerar todo el trabajo que hay que hacer, **por miedo a que si el valor es más alto el cliente lo rechace.**

Sobreestimar, cuando me preguntan por ejemplo cuantas horas me va a llevar un trabajo, y, sabiendo que me va a llevar 2 horas, digo 4 **por las dudas pase algo en el medio, inflando muchísimo la complejidad del producto.**

Las estimaciones se tienden a confundir con compromisos o planificación. **Uno planifica basándose en la estimación, pero no son lo mismo.**

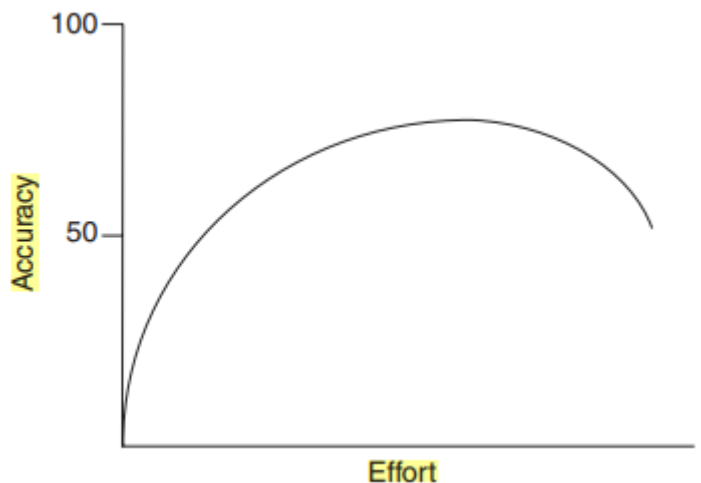
El gran beneficio del proceso de estimación es justamente “hacerlas”. Al estimar, se enriquece el trabajo del equipo, lo cual mejorará futuros procesos de estimaciones. Además, este proceso **ayuda a determinar la factibilidad del trabajo planificado** en etapas tempranas.

En el enfoque ágil, el esfuerzo de estimación es bajo porque la relación esfuerzo-precisión no es lineal:

Invertir demasiado tiempo no garantiza exactitud, mientras que un esfuerzo mínimo permite acercarse bastante a la realidad.

Por esta razón, el tiempo dedicado a estimar se define y asigna con anterioridad para no consumir más recursos de los necesarios.

El objetivo es situarse a la izquierda del gráfico, donde se obtienen altos niveles de certeza con una baja inversión de esfuerzo.



Existe una serie de consideraciones importantes:

Son medidas relativas, NO basadas en tiempo. Esto es debido a que las personas no saben estimar en terminos absolutos, sino comparando, lo cual es mas rapido.

Con esto obtenemos un mejor trabajo en equipo.

Permite enfocar el tiempo en entender el alcance y la complejidad de la US, en lugar de debatir horas exactas que vamos a tardar

Las estimaciones son flexibles y se corrigen sobre la marcha; lo que valida el avance es el software funcionando, no cumplir una predicción.

No se puede comparar una User Story con otra porque hay distintas dimensiones y ahí es donde entra el Story Point, que hace de unidad homogeneizadora que permite comparar.

¿Qué se estima?

Lo que se estima en ambientes ágiles, es solo el tamaño de las US (o Features) usando como unidad de medida los Story Points, y teniendo en cuenta 3 aspectos de estas: incertidumbre, complejidad y esfuerzo. Como la complejidad de las US tiende a incrementarse exponencialmente, se utilizan comúnmente series numéricas de esta índole, siendo una de las más comunes, la serie de Fibonacci.

Al ser estimaciones de tamaños relativos, (es decir, se estima realizando comparaciones de los tamaños de las US) lo importante en este proceso no es el valor que se asigna a cada US en particular, sino más bien que las estimaciones sean consistentes en el conjunto. Es decir, para un equipo, una US de 8 puntos puede no significar lo mismo que para otro equipo, pero lo que importa es que en un mismo equipo, una US de 5 puntos, siempre sea más chica que una de 8.

Es importante tener en claro que no se estima tiempo, ya que la duración de los Sprints es fija (concepto de Timebox).

Tamaño vs. Esfuerzo

Como el esfuerzo (horas de trabajo) es el factor más significativo del costo de un proyecto, tenemos una gran tendencia a confundir tamaño con esfuerzo y esfuerzo con calendario. La complejidad es independiente de quien haga el proyecto, y el esfuerzo depende específicamente de la persona que lo va a llevar a cabo lleva horas lineales

Un trabajo puede requerir un esfuerzo de x horas, pero no se puede asegurar que para tal día va a estar terminado.

Velocidad

Es una métrica del progreso de trabajo de un equipo. Se calcula (no se estima) sumando el número de Story Points asignados al estimar una US, que el equipo completa durante cada iteración, lo cual implica que la US se ha desarrollado y testeado por completo (DoD), y además el Product Owner la ha aceptado.

Si la US se completó parcialmente, no se cuentan sus Story Points en esa iteración (no existe el concepto de casi-completo, es una gestión binaria, es decir, se completó o no se completó la US). Esto genera que la velocidad del equipo baje en esa iteración, pero en la siguiente, aumentará, debido a que en unos pocos días probablemente se complete la US pendiente, y se pueda continuar con otras US planificadas en la iteración.

Esto implica que el equipo no se enfoque en la velocidad de cada iteración, sino más bien, en el promedio de velocidades a lo largo de sucesivas iteraciones. Esto brindará una mayor certeza acerca de la medida de progreso de trabajo del equipo.

Esta métrica permite retroalimentar el proceso de estimación, ya que un equipo maduro y con mucho trayecto, puede permitirse estimar cuántos SP puede desarrollar en la siguiente iteración a través del cálculo de velocidad.

Reestimación

Solo cambias los SP si descubres que es más grande o más chica en comparación con las demás.

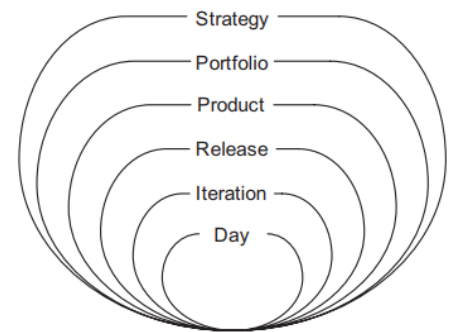
Nunca debes reestimar solo porque el equipo va despacio o tardó más de la cuenta: el tamaño de la tarea no cambió, lo que cambió fue el ritmo de trabajo.

Esa lentitud se refleja de manera automática en la velocidad del equipo con el paso de los sprints.

Planificación

En entornos ágiles, planificar consiste en ver periodicamente el contexto para aplicar ajustes para cumplir los objetivos.

El verdadero valor es el continuo análisis y adaptación, no es generar documentos largos; la documentación sirve como herramienta de apoyo y no es el objetivo final.

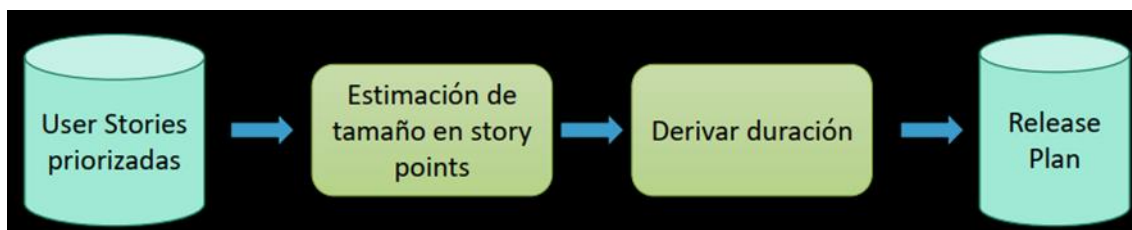


Los equipos ágiles afrontan esta situación, planificando al menos en 3 niveles:

Planificación de Release

Este proceso se da al inicio del proyecto, donde se analiza y determina qué US, épicas o Temas serán desarrollados en una nueva entrega (Release) del producto. Este plan se va actualizando mientras el proyecto transcurre (usualmente al comienzo de cada iteración), de manera que siempre refleja las expectativas actuales sobre qué se incluye en la entrega.

Una vez que se priorizan las US a implementar, se estima su tamaño con Story points, y teniendo en cuenta la velocidad de trabajo del equipo, se deriva la duración de la Release o del proyecto (la cantidad de sprints necesarios para obtener el release).



Planificación de Iteración

Este proceso se da al comienzo de cada iteración, donde el PO identifica aquellas US de mayor prioridad que deberán ser implementadas en la iteración, basándose en la iteración anterior ya finalizada. Esto se hace teniendo en cuenta el tiempo (que es una variable constante por ser metodología ágil) y la velocidad de trabajo del equipo (el cual se mantiene constante también).

Durante esta planificación, se discute sobre las tareas que serán necesarias para transformar una US en código funcional y testeado.

Planificación del día

Este proceso permite coordinar y sincronizar el trabajo del equipo, normalmente en reuniones diarias. Allí sólo se discuten las tareas a realizar durante el día, haciendo foco en las actividades que deberán ser desarrolladas para lograr cumplir con una tarea específica.

Tamaño

Dado que el tamaño es lo que se compara, lo definimos como **la medida de cuán compleja y grande es una US, además de cuánto trabajo es requerido para hacer** o completar una Feature/US. También el tamaño es función de la **incertidumbre de dicha US**, definida la incertidumbre como la falta de información.

una US de mismo tamaño puede implicar esfuerzos diferentes en función de la persona que se esté evaluando, debido a que el esfuerzo varía dependiendo de la persona

Formas de estimar el tamaño

Existen diferentes escalas para estimar el tamaño y siempre son por comparación, entre ellas:

1. **Número del 1 al 10.**
2. **Talles de remeras:** S, M, L: suele ser utilizado como escala de estimación para los ítems del Product Backlog.
3. **Serie exponencial de base 2:** 2, 4, 8, 16, 32, 64, etc.
4. **Serie de Fibonacci:** 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc.

Una vez definida la escala, no se cambia. En caso de cambiarla, también se debe cambiar el patrón de medición.

Story Point

Es una unidad de estimación que refleja la complejidad, incertidumbre y esfuerzo que le llevara al equipo completar una US, en comparación con otra.

La complejidad tiende a incrementarse exponencialmente.

- **Complejidad:** que tan compleja es implementar esa US. Relacionada a la cantidad de partes y relaciones entre las mismas posee la US.
- **Esfuerzo:** es importante que estime la persona que va a hacer el trabajo, ya que el esfuerzo es distinto entre dos personas. Son las horas persona lineales que se requieren para terminar la US.
- **Incetidumbre:** es cuánta información falta para ser más adecuados en la estimación.

De Negocio: es cuando todavía no entendemos del todo qué necesita el usuario o cómo debería funcionar la solución. La duda no está en el código, sino en las reglas del negocio. Es decir, es una duda del dominio del problema.

Tecnica: se refiere a tener claro qué quiere el usuario, pero dudar sobre las tecnologías para programarlo. Duda en el dominio de la solución.

¿Para qué se estima?

1. **En un primer momento para determinar cuántas Users se puede comprometer el equipo de trabajo a terminar en una iteración.**
2. **Al finalizar la iteración, contamos los puntos de historia correspondiente a las Users aceptadas por el Product Owner y obtenemos la velocidad del sprint**, la cual es una de las métricas fundamentales en ambientes ágiles.

Sin embargo, es necesario aclarar que la velocidad no es una estimación sino una medición propiamente dicha. Si un equipo alcanza una velocidad sostenible en el tiempo, es decir que mantiene la velocidad en las diferentes iteraciones, permite que el equipo obtenga previsibilidad, la cual ayuda a estimar en las siguientes iteraciones.

¿Cómo se estima la duración del proyecto?

Si la estimación se realiza utilizando Story Points para medir la complejidad relativa de las User Stories, para determinar la duración de un proyecto se realiza la derivación tomando el número total de story points de sus US y dividiéndolas por la velocidad del equipo.

Poker Planning o Poker Estimation

Es una técnica de estimación en ambientes ágiles

El equipo estima el esfuerzo que llevará realizar cada US, y cada integrante da su opinión al respecto, aportando su experiencia y conocimientos al resto del grupo.

El Product Owner participa en la planificación, pero no realiza estimaciones, suele ser el moderador

Escala

Dado que la complejidad en el software incrementa de forma exponencial, se decide adoptar la serie de Fibonacci como escala para la determinación de los puntos de historia.

Valor 1 o 1/2: son los valores más bajos y se aplican a una funcionalidad pequeña.

Valor 2-3: funcionalidad pequeña a mediana.

Valor 5: funcionalidad media.

Valor 8: funcionalidad grande, donde nos preguntamos si es posible dividir.

Valor 13 o más: necesariamente hay que dividir ya que no ingresa en el sprint.

Procedimiento

Se define una lista de actividades, módulos, casos de uso, Users stories, etc.

Se acuerda cuál de las anteriores es la que tiene menor grado de incertidumbre y se la define como canónica asignándole 1 o 2 story point (es la más simple). Esta User es utilizada como referencia para comparar.

Cada miembro elige una carta con el valor de la estimación del tamaño de la U.S. (comparado con la U.S. canónica y teniendo en cuenta los 3 aspectos nombrados) y se espera a que todos decidan.

Utilizando juicio experto, cada integrante del equipo de desarrollo expone su valor de estimación de forma simultánea, por medio de las cartas, dándolas vuelta a las que estaban boca abajo. En caso de que la estimación difiera, el mayor y el menor estimador justifican sus estimaciones para conocer sus opiniones. Luego el resto del medio aportan sus opiniones.

En caso de que la primera vuelta existiese diferencias considerables, se realiza nuevamente la ejecución

Si las estimaciones no convergen, entonces se llega a una estimación intermedia o promedio.

Consideraciones

1. Para realizar una estimación se debe tener en cuenta el criterio de Done y no solo la tarea de programación,
2. El tiempo para realizar las estimaciones, como cualquier otra actividad en los ambientes ágiles, es time boxing.
3. Las US por estimar son las menos posibles, en orden con el principio de diferir compromisos.

Ventajas

- Permite opiniones multidisciplinarias de todo el equipo, lo cual enriquece la estimación.
- Al generar debates y discusiones, permite lograr una mejor comprensión del dominio en los miembros del equipo.
- Estudios demuestran que reunir estimaciones individuales conducen a mejores resultados, que estimar de forma grupal.
- Es una actividad que resulta divertida para el equipo.

Estimaciones en Lean

En Lean, el cual es más extremo que las metodologías ágiles, asume que las estimaciones son opcionales y en algunas ocasiones pueden convertirse en un desperdicio dado que tienen una alta probabilidad de no coincidir con la realidad, dado que cualquier variable que se modifica, modificara a la estimación también.

Estimaciones en proyectos pequeños

Para estimar proyectos pequeños lo mejor es utilizar datos históricos propios de la organización, dado que en estos casos los datos históricos de la industria no se adaptan correctamente a estas situaciones. Las estimaciones en proyectos pequeños dependen en gran medida de las capacidades de los individuos que hacen el trabajo. El SEI propone el Personal Software Process para proyectos unipersonales o 2 personas, conocidos como "mini proyectos".

Diferencias con las estimaciones en ambientes tradicionales

1. Respecto a las tres dimensiones

En Agile se tiene fijo el tiempo (lo que dura la iteración) y los recursos afectados para trabajar, dejando variable el alcance. Por lo tanto, al estimar en estos ambientes se responde a cuánto se puede comprometer el equipo, que cantidad de las características de producto se puede construir en la iteración. Se entiende, además, que en estos ambientes el producto no está completamente definido, sino que se tiene una visión de este.

En los ambientes tradicionales, el alcance es lo que está fijo y el producto se encuentra definido. A partir de esto, se deriva el resto de las variables. Por esta razón en los procesos definidos se estima primero el tamaño, ya que es la base para estimar el resto.

2. Respecto a la precisión

Las estimaciones en los ambientes tradicionales buscan ser más precisas, debido a que son las bases para luego planificar. Dado que la precisión es realmente cara, la precisión en los ambientes ágiles y en correspondencia con la filosofía Lean, es considerada un desperdicio.

3. Principal diferencia

Las estimaciones ágiles son relativas, a diferencia de las estimaciones en ambientes tradicionales que son absolutas. En Agile, las características del producto son estimadas utilizando una medida de tamaño relativa dado que las personas no somos buenas realizando estimaciones absolutas, sino que es más fácil y rápido comparar medidas. Por otra parte, la práctica de estimar mediante comparaciones permite obtener una mejor dinámica grupal y pensamiento de equipo por sobre el individuo.

4. Quien estima

En las metodologías de estimación tradicional, las estimaciones las realiza el **líder del proyecto** o en ocasiones las realiza un **experto**, por lo que su estimación no es representativa de la cantidad de trabajo que requiere una persona del equipo para terminar una funcionalidad del producto. Esta es una de las razones principales de los desvíos en los proyectos de software tradicional. En los ambientes ágiles y en concordancia con los valores y principios explicitados en el manifiesto ágil, es el **equipo** el que realiza la estimación. Lo óptimo es que cada uno estime su propio trabajo y de esta manera, es posible disminuir la probabilidad de desviaciones.

5. Confusión con la planificación

En los ambientes tradicionales existe un gran error que es asumir las estimaciones como compromisos, como si fueran planificaciones. Esto implica que las estimaciones se transforman en promesas.

Mientras que en agile, asumen que las estimaciones están inmersas en el campo de las probabilidades y por lo tanto no implican un compromiso. Una falla en la estimación no implica extender el tiempo, ya que en ágiles trabajamos con el tiempo fijo, por lo tanto, lo que se ajusta es el alcance.

6. Momentos para estimar

En el enfoque tradicional se estima al comienzo del proyecto, luego de la especificación de requerimientos y luego del diseño (debido que ahí es cuando puedo saber cuál es el tamaño y de ahí derivo todas las demás estimaciones). Realmente se estima cada vez que se modifica el alcance, ya que es la variable que se mantiene fija y de la cual se deriva el resto.

Las estimaciones a lo largo del proyecto se van haciendo cada vez más certeras. Al comienzo, pueden diferir hasta un 400%, esto se debe a:

1. Al comienzo tenemos poca información y alta incertidumbre, por lo que aumenta el riesgo de estimar erróneamente.
2. Las estimaciones son demasiado optimistas.
3. Decimos que aquel que no sabe estimar implica estimar muchas veces.

En ágiles se estima al comienzo de cada sprint.

Frameworks de SCRUM a nivel de equipo y escala

SCRUM

Scrum es un marco ligero que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptables para problemas complejos.

En pocas palabras, Scrum requiere un Scrum Master para fomentar un entorno donde:

1. Un propietario del producto (Product Owner) ordena el trabajo de un problema complejo en un Product Backlog.

2. El equipo de Scrum convierte una selección del trabajo en un Incremento de valor durante un Sprint.
3. El equipo de Scrum y sus partes interesadas (stakeholders) inspeccionan los resultados y realizan los ajustes necesarios para el próximo Sprint.
4. *Repetir*

Scrum es una forma de trabajar en equipo que no te da un manual paso a paso, sino las reglas mínimas para que el grupo se organice solo y resuelva problemas sobre la marcha.

En vez de seguir planes, se aprende de la experiencia sobre la marcha (empirismo) y se eliminan desperdicios para centrarse únicamente en lo importante (Lean).

El corazón de este método es el Sprint, un bloque de tiempo fijo que contiene las reuniones clave para cumplir los pilares del empirismo. Es un ciclo corto que se repite una y otra vez (iterativo), lo que permite entregar partes funcionales del producto poco a poco (incremental).

El Equipo Scrum es la unidad base: un grupo pequeño, sin jerarquías ni subequipos, enfocado en un único Objetivo del Producto. Incluye al Scrum Master (SM), Product Owner (PO) y Desarrolladores.

Es multifuncional, ya que posee todas las destrezas necesarias, y autogestionado, porque decide internamente quién, cómo y cuándo trabajar.

Su tamaño ideal es de **diez personas o menos** para mantener **agilidad y buena comunicación**; si crece demasiado, debe **dividirse en varios equipos** que compartan el mismo producto, Objetivo, PO y Product Backlog (PB).

Se encargan del **producto** de **principio a fin**. Tienen libertad para organizar sus tareas a un ritmo de trabajo sano, con la única obligación de entregar un **incremento útil** al terminar cada Sprint.

Los Desarrolladores se comprometen a construir un incremento funcional en cada ciclo. Sin importar su especialidad, **son responsables de crear el plan del Sprint**, asegurar la **calidad cumpliendo** la Definición de Hecho, ajustar su plan diario para **lograr el Objetivo del Sprint** y exigirse **responsabilidad mutua**

El PO busca maximizar el valor del producto y es el único responsable de gestionar el PB.

Sus tareas incluyen definir y comunicar el Objetivo del Producto, crear y ordenar los elementos del PB, y asegurar que este sea visible y claro.

Puede delegar estas tareas, pero mantiene la responsabilidad final. El PO representa a los interesados, quienes deben negociar con él si desean cambios.

Para que tenga éxito, **toda la organización debe respetar sus decisiones**

El SM es un líder responsable de que Scrum se entienda y aplique correctamente,

Apoya al equipo **capacitándolo** en autogestión y multifuncionalidad, eliminando obstáculos, guiando el foco hacia **Incrementos de valor** y asegurando que los **eventos** se realicen en el **tiempo** fijado.

Asiste al PO enseñándole técnicas para gestionar el PB, logrando que el equipo entienda los requisitos y facilitando la relación con los interesados.

Finalmente, sirve a la organización **liderando la adopción de Scrum**, enseñando el **enfoque empírico** y **conectar** a los **equipos** con toda la **organización**

Eventos de Scrum

El Sprint es un contenedor para todos los eventos. Cada evento en Scrum es una oportunidad formal para inspeccionar y adaptar los artefactos de Scrum.

Los eventos se utilizan en Scrum para crear regularidad y minimizar la necesidad de reuniones no definidas en Scrum.

Timebox

Es un intervalo de tiempo fijo e inamovible asignado para realizar un evento dentro de Scrum. Sirve para evitar el desperdicio de tiempo y mantener un ritmo de trabajo sostenible.

En el marco de la triple restricción, su propósito es forzar a que siempre se negocie sobre el alcance del producto y nunca sobre el tiempo o el costo.

Si al finalizar el sprint no se llega con lo previsto, el tiempo jamás se extiende, sino que se reduce el alcance entregando menos funcionalidad.

Todos los eventos son un timebox y tienen una duración máxima establecida. La única excepción en ocurre con el refinamiento del Product Backlog: al ser una actividad continua a lo largo del desarrollo, no cuenta con un bloque de tiempo fijo definido.



Definición de Listo (DoR)

El DoR es un criterio que define cuando una US está lo suficientemente bien formulada como para poderse incluir en un Sprint. Por ejemplo, que cierta US debe tener un prototipo, entonces cuando el prototipo esté realizado, el DoR dirá "se ha definido el prototipo para la US".

El DoD es un criterio que dice cuando una historia está lo suficientemente bien implementada como para entrar al Sprint Review y ser mostrada al PO, entonces él será el encargado de aprobar o no la US y en caso de que esté aprobada decidirá si desea ponerla en producción.

En el DoD se arma un checklist con todo lo que debe cumplir una US para entrar en producción.

Definición de Hecho (DONE)	
☐	Diseño revisado
☐	Código Completo
☐	Código refactorizado
☐	Código con formato estándar
☐	Código Comentado
☐	Código en el repositorio
☐	Código Inspeccionado
☐	Documentación de Usuario actualizada
☐	Probado
☐	Prueba de unidad hecha
☐	Prueba de integración hecha
☐	Prueba de sistema hecha
☐	Cero defectos conocidos
☐	Prueba de Aceptación realizada
☐	En los servidores de producción

Capacidad del equipo en un Sprint

La capacidad es una **métrica que utiliza SCRUM para determinar cuánta cantidad de trabajo puede comprometer un equipo para un determinado Sprint.**

La capacidad se estima y se mide.

Una de las cosas que se hace en el Sprint Planning es **determinar la capacidad del equipo, y entonces, teniendo en cuenta la capacidad, se contrasta en cuántas US del Product Backlog se pueden incorporar en el Sprint Backlog.**

Para equipos más maduros, la capacidad puede ser estimada en Story Points y para equipos menos maduros se puede estimar en horas ideales.



Niveles de Planificación

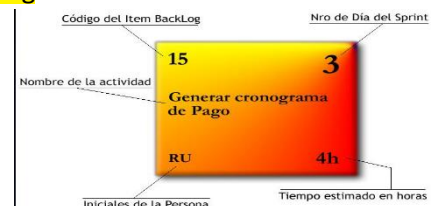
Nivel	Horizonte	Quién	Foco	Entregable
Portfolio	1 año o más	Stakeholders y Product Owners	Administración de un Portfolio de Producto	Backlog de Portfolio
Producto	Arriba de varios meses o más	Product Owner y Stakeholder	Visión y evolución del producto a través del tiempo	Visión de Producto, roadmap y características
Release	3 (o menos) a 9 meses	Equipo Scrum entero y Stakeholders	Balancear continuamente el valor del cliente y la calidad global con las restricciones de alcance, cronograma y presupuesto	Plan de reléase
Iteración	Cada iteración (de 1 semana a 1 mes)	Equipo Scrum entero	Que aspectos entregar en el siguiente Sprint	Objetivo del Sprint y Sprint Backlog
Día	Diaria	Scrum Master y Equipo de desarrollo	Cómo completar lo comprometido	Inspección del progreso

Herramientas de SCRUM

Tarjeta de tarea

Es una tarjeta que cuenta con 5 partes:

1. **Código del Ítem Backlog:** es un identificador del ítem en el Product Backlog.
2. **Nombre de la actividad:** suele ser una frase verbal que define qué es lo que hay que hacer.
3. **Iniciales del responsable de realizar la tarea.**
4. **Número de día del sprint.**
5. **Estimación del tiempo para finalizar la tarea.**



Tablero

Tablero utilizado por el Equipo de Desarrollo en el cual se colocan las tarjetas de tareas a realizar durante el sprint.

Se encuentra dividido en 5 secciones:

1. **Story:** el nombre de la historia.
2. **To Do:** aquí se encuentran las tareas por hacer en el Sprint.
3. **Work In Process:** aquí se encuentran las tareas que se están realizando.
4. **To Verify:** aquí se encuentran las tareas finalizadas que deben verificarse.
5. **Done:** aquí se encuentran las tareas terminadas: finalizadas y verificadas.

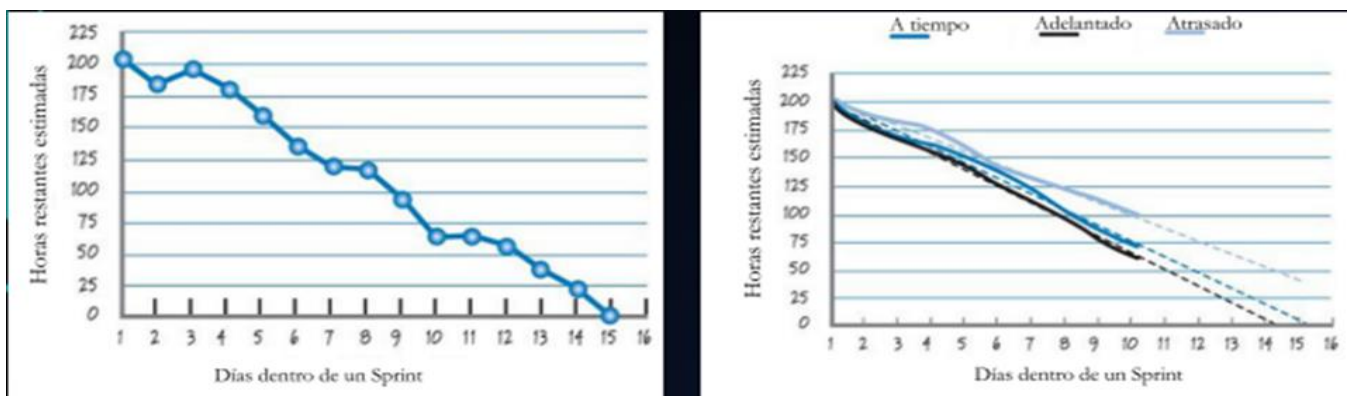
Lo ideal es que el nivel de granularidad de las tareas sea lo más fino posible, de tal forma que se detalle profundamente el avance del sprint, teniendo varias tareas en cada sección del tablero.

Gráficos del Backlog

Gráficos que brindan información acerca del progreso de un Sprint, de una release o del producto. El backlog de trabajo es la cantidad de trabajo que queda por ser realizado (en horas), mientras que la tendencia del backlog compara esta cantidad con el tiempo (medido en días).

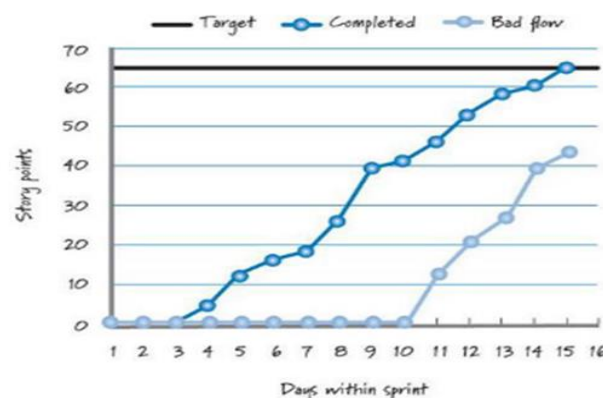
Sprint Burn-Down Chart

Visualiza con una curva descendente cuánto trabajo queda para terminar. En el eje de las abscisas se coloca el tiempo que va desde el inicio del sprint. En el eje de las ordenadas se coloca los puntos de historia a realizar en el sprint.



Sprint Burn-Up Chart

Visualiza con una curva ascendente cuánto trabajo se ha completado a lo largo de la release, es decir que visualiza los puntos de historia terminados en cada sprint.



Combined Burn Chart

Visualiza cuánto trabajo queda para terminar y cuanto trabajo se ha realizado.



Evolución de los productos de software

La pirámide de desarrollo de software es un modelo que describe los niveles de madurez y valor que alcanza un producto digital. Sirve para orientar el diseño y la construcción del software, asegurando que no solo cumpla requisitos técnicos básicos, sino que también genere un impacto positivo en la vida del usuario.

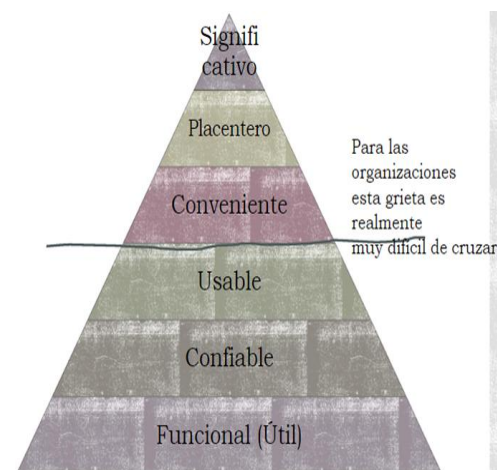
En su base se encuentra lo funcional, que define qué tareas debe realizar el sistema para resolver una necesidad inicial.

A continuación se sitúa la confiabilidad, la cual hace que los usuarios no corran peligro usando el producto y puedan confiar en los resultados.

El siguiente nivel es la usabilidad, asociada a UX/UI, cuyo objetivo es lograr que la interacción sea intuitiva y agradable; esto exige que el software sea productivo y ahorre tiempo real, no solo que permita completar una acción.

Por encima de estos niveles existe una línea divisoria que da paso a la conveniencia, el placer y el valor significativo.

La mayoría de los productos no superan esta barrera, ya que requiere que el sistema pueda ofrecerle una solución capaz de simplificar sus actividades y transformar su forma de trabajar o vivir.



Planificación del producto ágil

En las metodologías ágiles, se tiene una visión del producto reflejado en el Product backlog, que contiene todas las características de la funcionalidad que requiere el producto que han sido identificadas hasta el momento. Esta visión se descompone en releases, las cuales son las entregas frecuentes que se hará al cliente. Cada release se

construye a partir de sucesivas iteraciones.

Valor vs desperdicio

La productividad de un Startup no puede medirse en términos de cuánto se construye cada día, por el contrario, se debe medir en términos de construir lo correcto cada día y lo correcto es aquello que agrega valor. Si no construimos valor, entonces estamos invirtiendo esfuerzo en desperdicio.

Mínimo Producto Viable

Es un concepto de Lean Startup que enfatiza el impacto del aprendizaje en el desarrollo de nuevos productos. Se define como **la versión de un nuevo producto que permite a un equipo recopilar la cantidad máxima de aprendizaje validado sobre los clientes con el menor esfuerzo.**

A este concepto le subyace la idea de que ver lo que la gente realmente hace con el producto es mucho más confiable que preguntarle a la gente qué harían.

Una premisa clave detrás de la idea del MVP es que usted produce un producto real que puede ofrecer a los clientes y observar su comportamiento real con el producto o servicio.

Entre sus **características** se destaca que:

1. **Tiene el valor suficiente para que las personas estén dispuestas a usarlo o comprarlo.**
2. **Demuestra beneficios a futuro para retener a los usuarios.**
3. **Proporciona un ciclo de retroalimentación para guiar el desarrollo a futuro.**

Los errores comunes son:

1. Confundirlo con MMF o MMP. **La MMF (Característica Comercializable Mínima) o el MMP (Producto Comercializable Mínimo) a diferencia del MVP se enfocan en ganar, en cambio un MVP se enfoca en el aprendizaje**
2. **Enfatizar la parte de mínima y dejar de lado la parte de viable. Si el producto no tiene la calidad suficiente, no será posible obtener la retroalimentación del cliente.**
3. **Hacer del MVP un producto final, es decir, entregar lo que consideran un MVP, y luego no hacer más cambios a ese producto, independientemente de los comentarios que reciban al respecto.**

Un MVP permite validar diferentes hipótesis:

1. **Hipótesis del valor:** prueba si el producto realmente está entregando valor a los clientes después de que comienzan a usarlo (ver si el producto es realmente el producto correcto). Una métrica de prueba: tasa de retención (más de la mitad de los usuarios volvieron a utilizar el sistema todos los días, solo un 25% dejó de usarlo).
2. **Hipótesis de crecimiento:** prueba cómo nuevos clientes descubrirán el Producto (ver si más personas desean usar el producto). Una métrica de prueba: tasa de referencia o Net Promoter Score (NPS) (Se corrió la voz acerca de lo que era, no fue siquiera necesario el uso de políticas de marketing o publicidad).



Preparar un MVP

- 1) Encontrar un nicho de mercado.
- 2) Roadmap: definición de muy alto nivel de qué características del producto se podrían tener a cierta altura del año. Esto **sirve para los inversores**.
- 3) Investigar la competencia: ver hasta dónde llega, qué hace, qué fortalezas tiene.
- 4) Pre-vender el MVP.
- 5) Testear las distintas suposiciones.
- 6) Asegurarse de que el MVP esté resolviendo el problema correcto.
- 7) Enfocarse solo en las funcionalidades principales.

Design Thinking

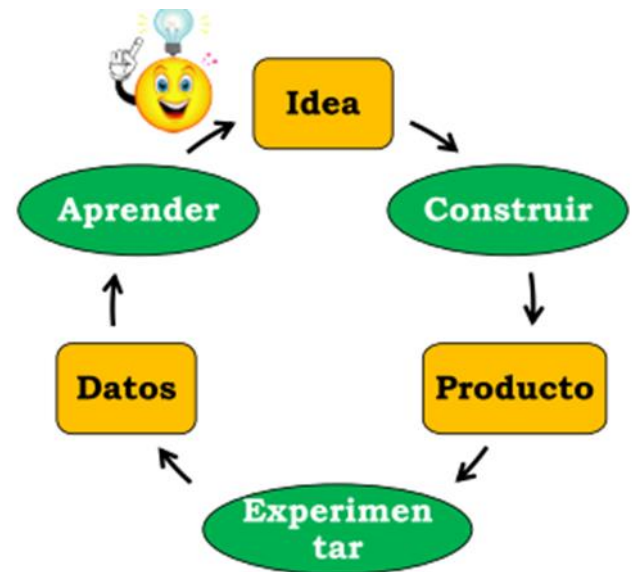
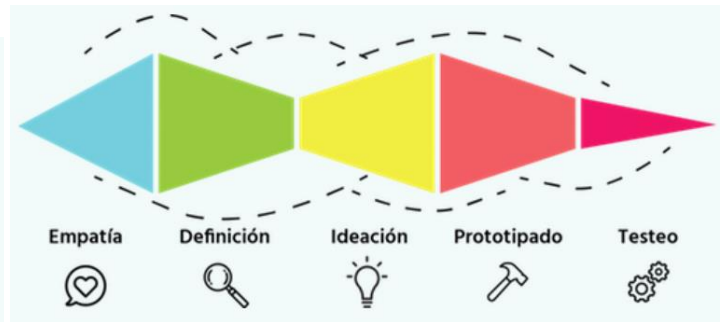
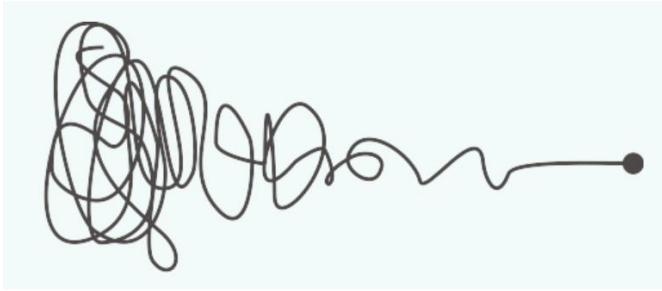
Es una metodología para la creación de servicios digitales centrada en el cumplimiento de las expectativas del usuario mediante un proceso de mejora continua que advierte las verdaderas necesidades de este.

Para llevar a cabo su proceso, utiliza un esquema de **pensamiento divergente y convergente**. Con el primero se busca tener una visión amplia del espectro de soluciones que responden al problema inicial. El segundo consiste en reducir a la mejor idea el conjunto de posibles soluciones, con el objetivo de brindar aquella que es más adecuada para las necesidades del cliente.

El proceso cuenta con 5 pasos y es iterativo (no lineal y se puede volver a los pasos anteriores las veces que sea necesario):

1. Empatizar: implica comprender las necesidades del cliente mediante la utilización del razonamiento intuitivo.
2. Definir: determinar lo valioso para el cliente a partir de lo recopilado durante la primera etapa
3. Idear: articular muchas soluciones posibles. Esto se puede lograr en los equipos multifuncionales, donde cada miembro aporta una mirada diferente para generar muchas ideas para luego filtrar las más viables.
4. Prototipar: consiste en materializar las ideas. Las ideas seleccionadas se transforman en prototipos que permitan visualizar la solución propuesta.

5. Probar: es una de las etapas más importantes del proceso, ya que permite validar los prototipos con los usuarios implicados en la solución, obteniendo retroalimentación y permitiendo iterar.

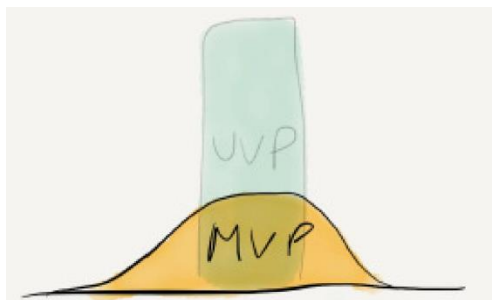


Explicación MVP, MVE, MMF con el dinosaurio Lean/Agile

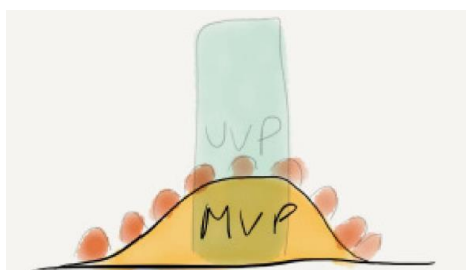
1. Comprender que un producto nuevo tiene una hipótesis de valor único: el Producto será único (UVP).



2. El siguiente paso es crear un producto mínimo viable (MVP) para probar su hipótesis. Esto se centra en su propuesta de valor única, pero normalmente también proporciona un poco de funciones de "Tablestakes" solo para asegurarse de que sea "viable" como producto.



3. Tu MVP también es una hipótesis. Podría ser lo suficientemente bueno para encontrar un mercado o no. Se muestra el caso en el que cada cliente potencial con el que interactúas te dice "Esto es genial, pero para poder usarlo necesito X" y X es diferente para cada cliente / usuario. Esto muestra que aún no se encuentra un mercado para el producto.

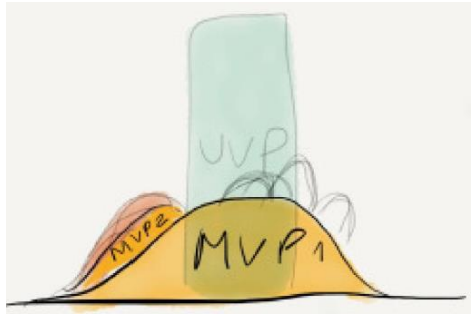


4. Si por el contrario ves cada vez más respuestas apuntando al MISMO X entonces tiene sentido revisar la hipótesis de Cliente/Problema/Solución. Básicamente, estás ejecutando un pivot. Está construyendo MVP2 centrado en la nueva hipótesis basada en el aprendizaje reciente de Desarrollo de clientes generado por el anterior MVP.

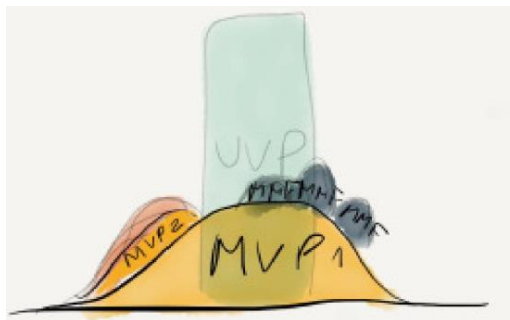


5. Supongamos que MVP2 es exitoso y está viendo una atracción real de los primeros usuarios. Si desea aumentar el crecimiento se busca que esos usuarios iniciales adopten todavía más el producto y, a la vez, empezar a sumar clientes del mercado general. Con el feedback y el análisis del producto, surgen áreas de oportunidad para crecer:
algunas permiten ampliar la propuesta de valor y otras hacen que el producto actual sea más robusto.

Área (o área de características): Es un módulo o sección específica del producto que agrupa funciones relacionadas (por ejemplo: pagos, catálogo o notificaciones) donde el equipo decide experimentar o agregar mejoras para hacerlo crecer.



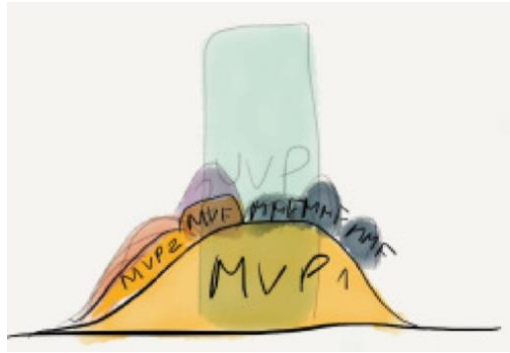
6. En el caso de áreas con una fuerte indicación de valor, puede directamente definir un MMF (Características mínimas comercializables), para encontrar la pieza mínima que pueda empezar a traer crecimiento. El objetivo del MMF es aportar valor. Supone una alta certeza de que existe valor en esta área y que sabemos cuál debe ser el producto para proporcionar este valor. La razón para dividir una característica grande en MMF más pequeños es principalmente el tiempo de comercialización (Time to market) y la capacidad de aportar valor en muchas áreas. Una indicación de que está trabajando en MMF es que cuando al liberarse uno se siente cómodo trabajando en el próximo MMF en esa área.



7. Si se desea esperar para ver si el primer MMF apesta ... entonces está de vuelta en la tierra de la hipótesis. Ahora tu hipótesis se centra en una característica en lugar del producto. Tienes un área con alto potencial pero también alta incertidumbre.



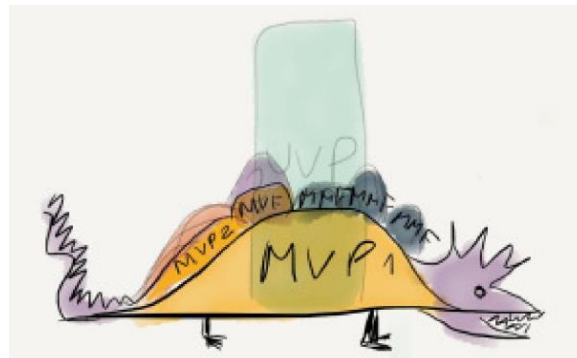
8. La forma de afrontarlo es crear una función "pionera": MVF (Característica Mínima Viable). La característica mínima que aún puede ser viable para uso real y aprendizaje de los usuarios reales. Si el MVF resulta exitoso (hit gold), puede desarrollar más MMF en esa área para tomar ventaja (si eso tiene sentido). Si no es así, puede cambiar a otro enfoque hacia esa área de características, o en algún momento buscar una ruta de crecimiento alternativa. Esencialmente, el MVF es una versión mini del MVP.



9.

En mercados inciertos, el producto se desarrolla probando distintos MVP. Una vez logrado el encaje con el mercado, se combinan MMF y MRF según el nivel de incertidumbre de cada área. Aunque un MVP, MMF o MRF es la unidad mínima indispensable para el negocio, su desarrollo técnico puede ser demasiado grande. Por eso, se dividen en piezas más pequeñas llamadas User Stories para reducir riesgos técnicos y de ejecución. Estas partes pueden tener o no un valor comercial directo por sí solas, pero permiten obtener retroalimentación temprana y ajustar decisiones a lo largo del camino.

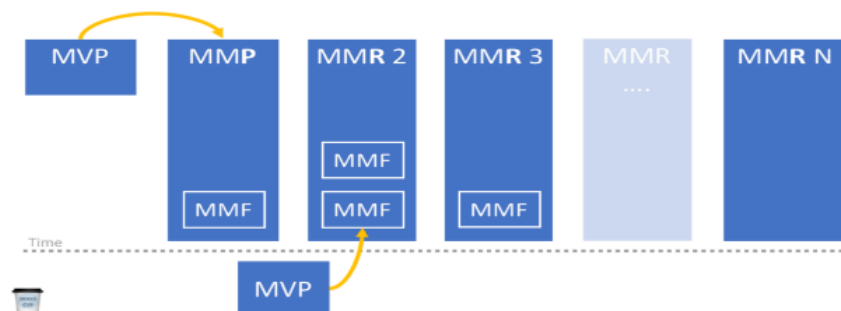
MVF: Alta incertidumbre, no sabemos si al usuario le va a interesar la función.
Objetivo = Aprender



MMF: Alta certeza. Los usuarios necesitan la función y aporta valor. Se construye una porción chica para lanzarla rápido al mercado.
Objetivo = Valor comercial.

Productos mínimos para la gestión de productos

1. Producto Mínimo Viable (MVP). Tanto esta como el MVP responden a hipótesis.
2. Característica Mínima Viable (MVF), es una versión mini del MVP.
3. Característica Mínimas del Release (MRF). Son las características mínimas debe tener un producto para salir al mercado. Es una versión del producto que se publica y que el cliente puede usar.
4. Característica Mínima Comerciable (MMF).



Unidad 3: Gestión del Software como producto

Conceptos introductorios de la gestión de configuración

Introducción

Los sistemas de software siempre cambian durante su desarrollo y uso. Conforme **se hacen cambios al software, se crean nuevas versiones del sistema**. Los sistemas pueden considerarse como un conjunto de versiones donde cada una de ellas debe mantenerse y gestionarse. Esto es necesario para no perder la trazabilidad de los cambios que se incorporan a cada versión.

Definición de Gestión de Configuración

Es una disciplina de soporte, transversal a todo el proyecto con aplicación en diferentes disciplinas, **que surge para enfrentar la crisis de problemas de software, aplicando dirección y monitoreo** (administrativo y técnico) a:

- **Identificar** características técnicas y funcionales de ítems de configuración;
- **Documentar** características técnicas y funcionales de ítems de configuración;
- **Controlar los cambios** de esas características identificadas y documentadas;
- **Registrar y reportar** estos cambios;
- **Verificar correspondencia** con los requerimientos (trazabilidad).

Su propósito es resolver problemas de diferente índole, a través del establecimiento y el **mantenimiento de la integridad** del producto de software a lo largo de todo su ciclo de vida.

Algunos de esos problemas son:

- Pérdida de componentes;
- Pérdida de cambios (la versión del componente no es la última);
- Regresión de fallas;
- Doble mantenimiento;
- Superposición de cambios;
- Cambios no validados.

Su propósito es establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida. **Esto implica identificar la configuración en un momento dado, controlar sistemáticamente sus cambios y mantener su integridad y origen.**

La integridad nos ayuda a garantizar que el producto a entregar tiene la calidad correspondiente.

Y nos garantiza un nivel mínimo de confiabilidad.

El problema de la calidad es que es subjetiva y se suma a que el software es intangible, por lo tanto, es muy difícil medir si la calidad que responde al cumplimiento de las expectativas del cliente realmente se verifica.

La idea de la integridad del producto es hacer explícita las características o expectativas que tiene el cliente sobre el producto de software.

Decimos que **se mantiene la integridad** de un producto de software **cuando:**

- **Satisface las necesidades de usuario:** hace lo que el usuario espera que haga;

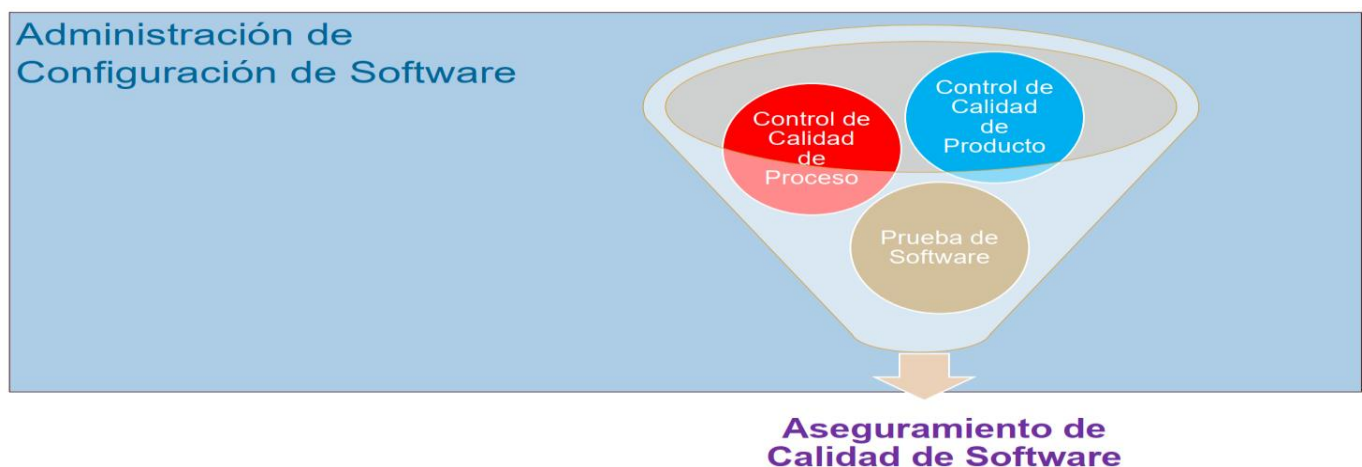
- **Permite la rastreabilidad durante todo su ciclo de vida:** existen vínculos o conexiones entre los ítems de configuración (los cuales deben ser definidos desde un primer momento del ciclo de vida del producto), que permiten analizar en donde impactará un cambio en un ítem de configuración. De esta forma, se puede determinar cómo impactará cada cambio de requerimientos a través de la trazabilidad. A mayor cantidad de vínculos → mayor información → mayor trazabilidad → mayor costo (analizar la relación costo-beneficio);
- **Satisface criterios de performance y RNF;**
- **Cumple con expectativas de costo:** este apartado incluye la satisfacción del equipo de proyecto. No solo el cliente debe estar satisfecho, sino también el desarrollo del producto debe ser redituable.

Mientras el producto existe hay actividad de gestión de configuración, que es responsabilidad de todo el equipo. Sin embargo, existen roles específicos como el rol del Gestor de configuración que tiene tareas adicionales como por ejemplo mantener la herramienta, marcar línea base, etc.

¿Cuál es el problema que se está tratando de solucionar?

El software es muy fácil de modificar, en el sentido de que uno puede entrar al repositorio y con un solo click eliminar meses de trabajo. Entonces lo que buscamos con la gestión de configuración del software es tratar de que estas cosas no pasen, buscando que el producto de software sea íntegro y si fuera el caso de que se van a realizar cambios, podamos tener un control de qué cosas cambiaron en un determinado momento. Es por esta razón que se asocia el concepto de software con el concepto de versión, ya que cada ítem de configuración debe tener su versión.

Y todas las personas se enteren de los cambios.



Conceptos Generales

Ítem de Configuración

Pieza que forma parte del producto, y pueden ser almacenados en un repositorio, sin importar su extensión. Pueden sufrir cambios, y se desea conocer su estado y evolución a lo largo del ciclo de vida (ya sea del producto o proyecto, dependiendo del artefacto). Es decir, es cualquier aspecto asociado al producto de software (requerimientos, diseño, código, datos de prueba, documentos, etc.) que es necesario mantener. Son todos aquellos elementos que componen toda la información producida como parte del proceso de ingeniería de software, como ser programas de computadora (código fuente y ejecutables), documentos que describen los programas (documentos técnicos o de usuario), datos (de programa o externos).

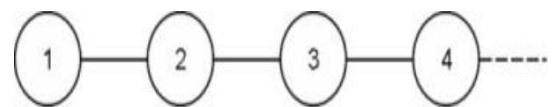
Los ítems, dependiendo de su naturaleza, pueden durar lo que dure el ciclo de vida del proyecto, producto o sprint.

Versión

Instancia de un ítem de configuración que difiere de otras instancias de este ítem. Las versiones se identifican unívocamente.

"La gestión de configuración permite a un usuario especificar configuraciones alternativas del sistema de software mediante la selección de las versiones adecuadas"; esto se puede gestionar asociando atributos a cada versión (que pueden ser datos sencillos como un nro. de versión asociado a cada objeto. Cada versión de software es una colección de elementos de configuración (ECS), como ser código fuente, documentos, datos).

La versión es un punto particular en el tiempo de ese ítem de configuración (es un estado).



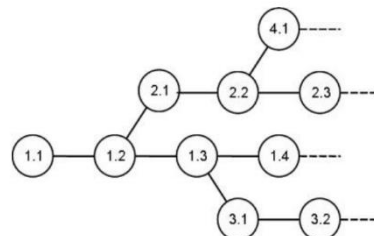
Evolución lineal de un ítem de configuración

Control de versiones

Hace referencia a la evolución de un único ítem de configuración o de cada ítem de configuración por separado.

Variante: Versión de un ítem de configuración que evoluciona por separado para satisfacer requerimientos en conflicto (SO, Hardware).

Se llama variante cuando las diferencias entre ambas versiones son mínimas, sin llegar a considerarse una versión completamente nueva.



Variante de un ítem de configuración

Configuración

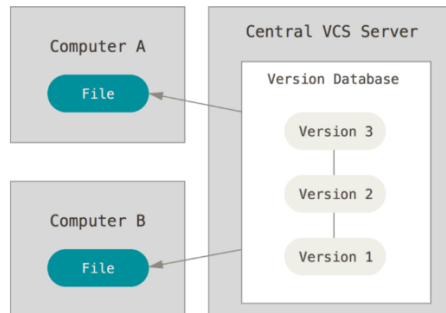
Conjunto de todos los ítems de configuración con su versión específica. Define una foto de los ítems de configuración en un momento de tiempo determinado.

Una configuración es el conjunto de todos los componentes fuentes que son compilados, sus documentos y la información de la estructura que definen una versión del producto a entregar. La configuración de un software es la sumatoria de todos los ítems de configuración que tiene en un momento determinado, equivale a una instantánea o una foto de todos los ítems de configuración con su versión en un momento del tiempo.

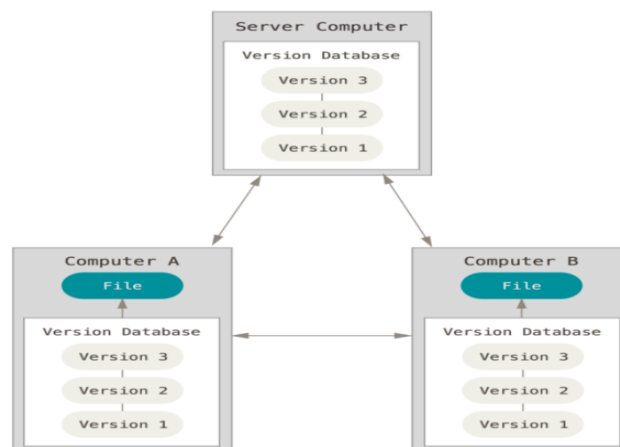
Repositorio

Es un contenedor de ítems de configuración, se encarga de mantener la historia de cada ítem con sus atributos y relaciones, además es usado para hacer evaluaciones de impacto de los cambios propuestos. Puede ser una o varias bases de datos. Tiene una estructura para mantener el orden y la integridad. El que tenga una estructura ayuda a la seguridad, los controles de acceso, las políticas de Backup y todo aquello que se aplica sobre un repositorio. Tenemos 2 tipos de repositorios:

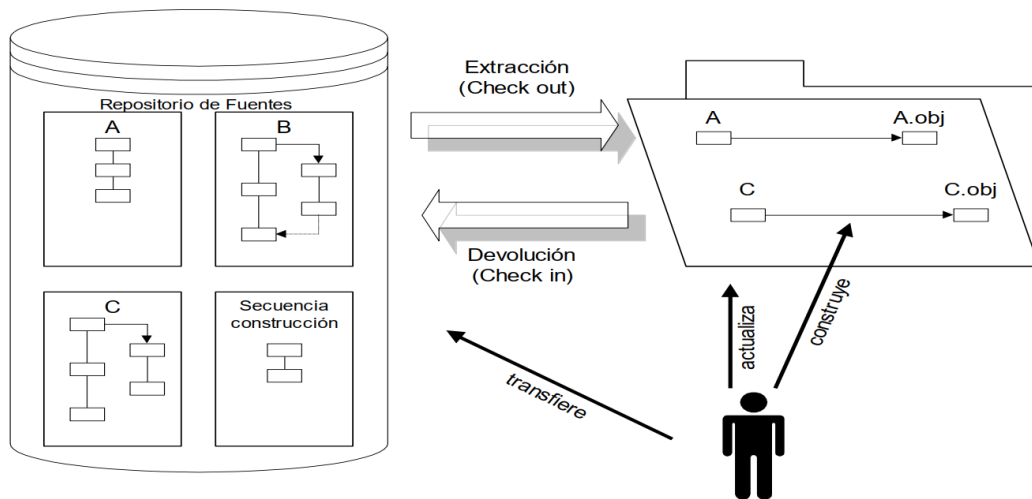
1. **Repositorios Centralizados:** está caracterizado porque un servidor contiene todos los archivos con sus versiones. Los administradores tienen un mayor control sobre el repositorio. Tiene como desventaja qué si falla el servidor, todo lo positivo se cae.



2. **Repositorios Descentralizados:** está caracterizado porque cada cliente tiene una copia exactamente igual del repositorio completo. Tiene como ventaja que se resuelve el problema de los servidores centralizados, debido a que si el servidor falla sólo es cuestión de “copiar y pegar”, además posibilita otros workflows no disponibles en el modelo centralizado. La desventaja que podemos mencionar es que es más complicado llevar un control sobre el mismo.



Funcionamiento de un repositorio



Rama (Branch)

Es un conjunto de ítems de configuración con sus correspondientes versiones, que permiten bifurcar el desarrollo de un software, por varios motivos:

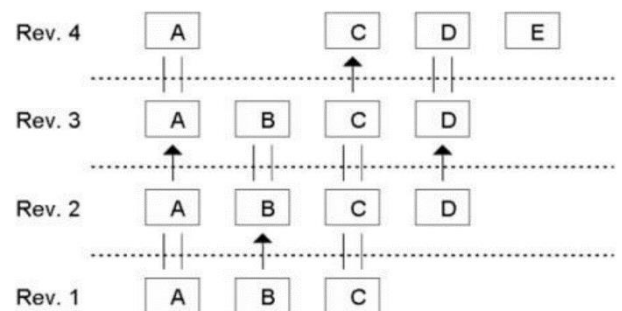
- Experimentación;
- Resolución de errores en el desarrollo;
- Desarrollar un mismo software para diferentes plataformas.

Integración de ramas (Merge)

Se da cuando se integran dos ramas, para fusionar la configuración de los ítems de configuración con sus correspondientes versiones, en cada una de ellas. Una buena práctica es mantener la rama principal como la versión estable, y fusionar los cambios hacia ella. En caso de no integrarse a la Main, las ramas deberían descartarse.

El rol de las líneas base y su administración

Es un conjunto de ítems de configuración que han sido contruidos y revisados formalmente, de manera que pueden ser tomados como referencia para demostrar que se ha alcanzado cierto nivel de madurez en ellos, y que sirve como base para desarrollos posteriores. Esto es lo mismo que decir que es una configuración de software que ha sido formalmente revisada y aprobada, que sirve como base para desarrollos futuros.



Este conjunto de ítems debe tener una referencia única, y esto se hace a través de "tags".

Se acuerdan parámetros para determinar qué se considera o qué debe cumplir los ítems de configuración para considerarse como línea base.

Si se desea cambiar una línea base, existe un protocolo formal de control de cambios, dirigido por el Comité de Control de Cambios, que permite definir el procedimiento a seguir para manejar peticiones de cambios, y en caso de aceptarse, acordar nuevamente los parámetros y comunicar los cambios a todo el equipo, para que tomen la nueva línea base como modelo a seguir.

¿Para qué sirve una línea base?

Fundamentalmente es para tener un punto de referencia, pero también sirve para hacer Rollback o saber qué se pone en producción. Nos permite saber cuál era la última situación estable en un momento de tiempo y cómo se fue evolucionando. Permiten ir atrás en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto.

Existen dos tipos de línea base:

1. **Operacionales:** contiene una versión de producto cuyo código es ejecutable, han pasado por un control de calidad definido previamente. La primera línea base operacional corresponde con la primera Release. Es la línea base de productos que han pasado por un control de calidad definido previamente.
2. **De especificación o documentación:** son las primeras línea base, dado que no cuentan con código. Podría contener el documento de especificación de requerimiento. Son de requerimientos, diseño.



Planificación de la gestión de configuración de software

Este plan se debe confeccionar al inicio de un proyecto, y existen diferentes estándares donde se expresa cómo planificar:

- Reglas de nombrado de los ítems de configuración;
- Herramientas para utilizar en SCM;
- Roles e integrantes del Comité de Control de Cambios;
- Procedimientos formales de cambios;
- Procesos de auditoría;
- Estructura del repositorio;

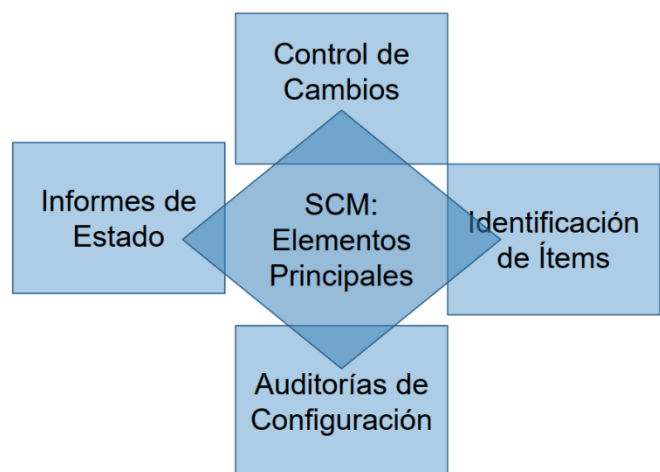
- SCM para software externo (opcional);
- Cómo se hará el control de cambios;
- Registros que deben mantenerse;
- Tipos de documentos.

Esta planificación se debe hacer temprano y definir documentos y procesos a administrar

Actividades relacionadas a la gestión de configuración

Conjunto de tareas que componen el Plan de Gestión de Configuración.

Se aplican en cualquier metodología de desarrollo, con la excepción de las auditorías, que no son compatibles con una metodología ágil pura.



Identificación de Ítems de Configuración

Consiste en darle a cada ítem una identificación única mediante reglas de nombrado y versionado, organizando la estructura del repositorio, la ubicación de cada ítem en él y los componentes de la línea base. Esto crea un camino que une todas las etapas del ciclo de vida del software, teniendo en cuenta el tiempo que es necesario mantener cada ítem.

Se clasifican en:

Ítem de producto: Elementos que duran toda la vida útil del software.

Ítem de proyecto: Elementos que duran lo que dura el proyecto.

Ítem de iteración: Elementos que duran solo un ciclo o sprint.

Luego de identificar los ítems se debe armar la estructura del repositorio con nombres representativos. Luego se vinculan los ítems de configuración con el lugar físico del repositorio donde se van a almacenar.

Control de cambios

Una vez definida la línea base, no es posible cambiarla sin antes pasar por un proceso formal de control de cambios. Es decir que el control se hace sobre los ítems de configuración que pertenecen a la línea base, ya que todos los trabajadores del software tienen a dichos ítems como referencia.

Comité de control de cambios

Este proceso es llevado a cabo por el comité de control de cambios, el cual se reúne para autorizar la creación y cambios sobre la línea base. Forman parte de dicho comité los interesados en evaluar y enterarse del cambio, decidiendo si lo aceptan o no. El líder del proyecto, el analista, el arquitecto e incluso el cliente pueden constituirlo. Realmente la integración del comité depende de la propuesta del cambio.

Etapas del proceso

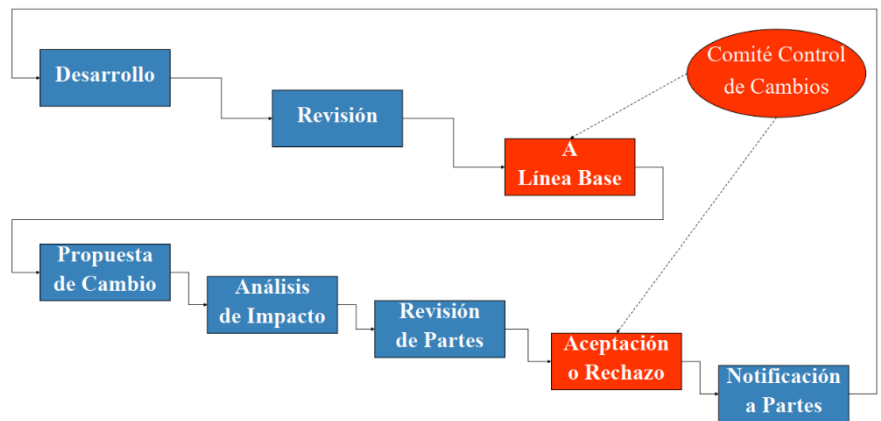
Si el comité fijó una línea base luego de la revisión de los ítems que la conforman y llegada a la conclusión de que los mismos se encuentran en estado estable, en caso de recibir una propuesta de cambio, los pasos son:

1. Se recibe una **propuesta de cambio** sobre una línea base determinada, no sobre un ítem.
2. El comité de control de cambios realiza un **análisis de impacto del cambio** para evaluar el esfuerzo técnico, el impacto en la gestión de los recursos, los efectos secundarios y el impacto global sobre la funcionalidad y la arquitectura del producto.

3. En caso de que se autorice la propuesta de cambio, se genera una orden de cambio que define lo que se va a realizar, las restricciones a tener en cuenta y los criterios para revisar y auditar.

4. Luego de realizado el cambio, el comité vuelve a intervenir para aprobar la modificación de la línea base y marcarla como línea base nuevamente, es decir que realiza una revisión de partes.

5. Finalmente se notifica a los interesados los cambios realizados sobre la línea base.



Además, el control de cambios permite tener una trazabilidad entre los ítems de configuración, y ante un cambio saber cuáles ítems están afectados por este.

Auditorías de configuración

Controles independientes y objetivos realizados durante el desarrollo por un auditor externo al equipo. Analizan las líneas base para ver el estado del software y si se cumple con las reglas de SCM.

Revisan si las tareas se hacen según el Plan de Gestión de Configuración y complementan a la revisión técnica, ayudando a la calidad e integridad.

Trazabilidad o rastreabilidad de requerimientos

Establecer una correspondencia de relación entre ítems de configuración de diferentes niveles de abstracción en el proceso de desarrollo de software. Una traza puede ser constituida por el requerimiento, sus modelos de análisis y de diseño, la implementación en código y los casos de prueba para validar y verificar el requerimiento. La trazabilidad permite realizar un análisis del impacto sobre el sistema a partir de una solicitud de cambio en el requerimiento. Los modelos de calidad exigen la trazabilidad bidireccional. Desde el requerimiento hacia el componente de código y viceversa.

Procesos a los que sirve

1. **Validación:** se encarga de asegurar que cada ítem de configuración resuelva el problema apropiado, es decir, lo que el cliente necesita.
2. **Verificación:** implica asegurar que un producto cumple con los objetivos definidos en la documentación de líneas base. Todas las funciones son llevadas a cabo con éxito y los test cases tengan status “ok” o bien consten como “problemas reportados” en la nota de reléase. Es decir, que se cumpla lo definido para cada ítem de configuración en la documentación de una línea base.

Las auditorías permiten visualizar si se están satisfaciendo los requerimientos y si se ha cumplido la intención de la línea base anterior. Ante un análisis, el auditor puede detectar defectos y ajustar correcciones.

Tipos de auditorías

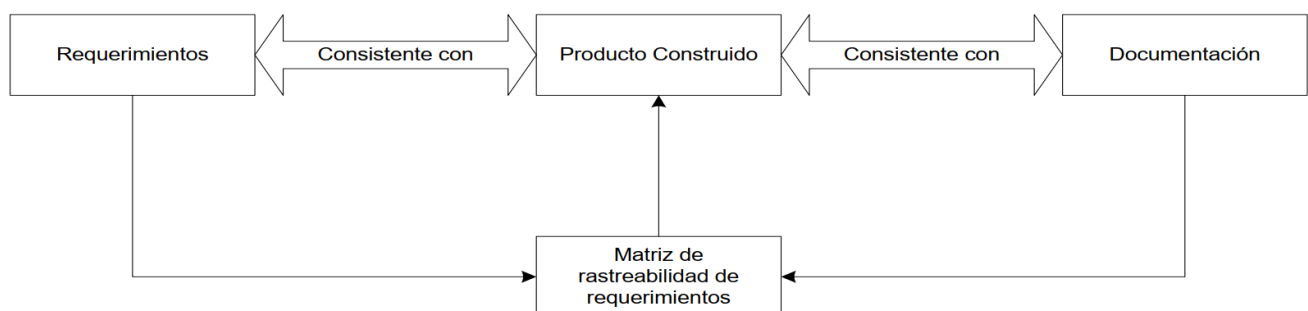
1. **Auditoría física o de contingencia:** se realiza primero. Se validan los ítems de configuración de la línea base contra el plan de gestión de configuración, asegurando que la documentación que se entregará es consistente con el código desarrollado. Si esto no pasa, la auditoría funcional no se realiza.
 - Controla que el repositorio esté alojado en el lugar donde se dijo que iba a estar, que los IC respeten el esquema de nombrado, que estén guardados donde se dijo que iban a estar guardados.
 - Lo que se audita es una línea base.
 - Hace verificación.
2. **Auditoría funcional de configuración:** audita la trazabilidad de requerimientos, controlando que la funcionalidad y performance real de cada ítem de configuración sean consistentes con la especificación de requerimientos. El propósito final es validar que el código implementa únicamente y todos los requerimientos de la ERS.

- Hace validación.

La gestión en ambientes ágiles se trabaja con integración continua (CI - Continuous Integration), un servidor donde se testea automáticamente el código y luego se integra en producción continuamente. Un repositorio, un esquema de nombrado son cosas que si se utiliza en ágiles, mientras que la auditoría y el comité de control de cambios durante el sprint, pueden obviarse.

Auditoría Funcional de Configuración

Auditoría Física de Configuración



Beneficios:

- Aumenta la protección contra cambios innecesarios;
- Mejora de la visibilidad de estado del proyecto y sus componentes;
- Aumenta la auto responsabilidad;
- Disminuye los costos por retrabajo;
- Disminuye el tiempo de desarrollo;
- Aumenta la calidad;
- Suministra viabilidad y rastreabilidad del ciclo de vida del producto de software.

Desventajas:

- Quejas (es burocrático, molesto, se meten con mi trabajo);
- La transición es difícil;
- No hay compromiso en todos los niveles;
- No hay consciencia del problema.

Informe de estado

proporciona un mecanismo para mantener un registro de cómo evoluciona el sistema, y dónde está ubicado el software, comparado con lo que está publicado en la línea base. Esto permite mantener a todo el equipo informado sobre la última línea base, y que se trabaje en base a su última versión, y no sobre una versión obsoleta.

Estos reportes incluyen todos los cambios que han sido realizados a las líneas base durante el ciclo de vida del software. Normalmente esto consta de grandes unidades de datos, por lo que se utilizan herramientas automatizadas por una computadora.

El objetivo principal es asegurarse que la información de los cambios llegue a todos los involucrados.

El reporte más conocido es el de inventario, el cual provee una copia del contenido del repositorio con la estructura de directorios.

Permite responder a preguntas como:

- a. ¿Cuál es el estado del ítem?
- b. ¿La propuesta de cambio fue aceptada o rechazada por el comité?
- c. ¿Qué versión de ítem implementa un cambio de una propuesta de cambio aceptada?
- d. ¿Cuál es la diferencia entre dos versiones de un mismo ítem?